CRICKET CLASH — DOC 2 OF 10

COMPLETE ENGINE ARCHITECTURE: 8 Core Engines That Power Everything

Event | Sync | Gameplay | Sound | Commentary | Lighting | Weather | Crowd

Repo: https://github.com/pyushmatania/handcricketgame-8ec46dab **Doc Scope:** This document covers ALL backend/logic systems — the "brains" behind the game. Every engine has full TypeScript architecture, configuration, integration points, and detailed behavior specs. UI rendering is in Doc 1. Sound/Commentary CONTENT (the actual lines and audio files) is expanded in Doc 3.

CHAPTER 1: ARCHITECTURE OVERVIEW

1.1 THE ENGINE PHILOSOPHY

Problem this solves: In the current app, events happen too fast, overlap, go out of sync between PvP players, commentary cuts out or plays at wrong times, sounds are inconsistent, and transitions feel rushed.

Solution: 8 independent engines connected via a central **Event Bus**. When something happens (a boundary, a wicket, an over ending), the Event Engine fires ONE event. ALL other engines hear it simultaneously and react independently. The Event Engine controls TIMING — nothing proceeds until the current event's mandatory duration expires.

Think of it like a film production:

- **Event Engine** = the Director (calls "Action!" and "Cut!")
- **Sync Engine** = the Script Supervisor (ensures both cameras/players see the same thing)
- **Gameplay Engine** = the Scriptwriter (determines what happens on each ball)
- **Sound Engine** = the Sound Designer (plays the right sounds at the right time)
- **Commentary Engine** = the Commentators (speak about what happened)
- **Lighting Engine** = the Lighting Director (flashes, glows, dimming)
- **Weather Engine** = the Set Designer (rain, dust, clouds)
- **Crowd Engine** = the Extras Director (crowd volume, reactions, waves)

1.2 DEPENDENCY GRAPH

```
                    ┌─────────────────┐
                    │   EVENT ENGINE  │ ← Central brain, everything flows through here
                    │  (Event Bus)    │
                    └─────────┬───────┘
                              │
            ┌─────────────────┼─────────────────┐
            │                 │                 │
            ▼                 ▼                 ▼
┌─────────────┐   ┌─────────┐   ┌──────────────┐
│   SYNC      │   │ GAMEPLAY │   │  UI/ANIM     │
```

```
│  ENGINE    │  │  ENGINE  │  │  (React)     │
│            │  │          │  │              │
│ Supabase   │  │ Ball     │  │ Framer       │
│ Realtime   │  │ Resolver │  │ Motion       │
└────────────┘  └──────────┘  └──────────────┘
        │             │             │
        ▼             ▼             ▼
┌────────────┐  ┌──────────┐  ┌──────────────┐
│   SOUND    │  │COMMENTARY│  │  LIGHTING    │
│  ENGINE    │  │  ENGINE  │  │  ENGINE      │
│            │  │          │  │              │
│ Howler.js  │  │ Web      │  │ CSS/Canvas   │
│            │  │ Speech   │  │ Overlays     │
└────────────┘  └──────────┘  └──────────────┘
        │             │
        ▼             ▼
┌────────────┐  ┌──────────┐
│  WEATHER   │  │  CROWD   │
│  ENGINE    │  │  ENGINE  │
│            │  │          │
│ Particles  │  │ Mood     │
│ System     │  │ System   │
└────────────┘  └──────────┘
```

1.3 PACKAGE DEPENDENCIES

```
{
  "dependencies": {
    "howler": "^2.2.4",           // Sound Engine — audio playback
    "zustand": "^4.5.0",          // Global state management for all engines
    "@tsparticles/react": "^3.0",  // Weather + celebration particles
    "tsparticles": "^3.0",        // Core particle engine
    "@supabase/supabase-js": "^2.101.1"  // Already present — for Sync Engine
  }
}
```

1.4 FILE STRUCTURE

```
src/
├── engines/
│   ├── EventEngine.ts          // Chapter 2
│   ├── SyncEngine.ts           // Chapter 3
│   ├── GameplayEngine.ts       // Chapter 4
│   ├── SoundEngine.ts          // Chapter 5
│   ├── CommentaryEngine.ts     // Chapter 6
│   ├── LightingEngine.ts       // Chapter 7
│   ├── WeatherEngine.ts        // Chapter 8
│   ├── CrowdEngine.ts          // Chapter 9
│   ├── types.ts                // Shared types for all engines
│   └── EngineManager.ts        // Initializes and connects all engines
├── stores/
│   ├── matchStore.ts           // Zustand store for match state
│   ├── engineStore.ts          // Zustand store for engine states
│   └── settingsStore.ts        // User preferences (volume, language, etc.)
├── data/
│   ├── commentary/
│   │   ├── english.ts          // 500+ English commentary lines
│   │   ├── hindi.ts            // 500+ Hindi lines
│   │   └── hinglish.ts         // 500+ Hinglish lines
│   └── sounds/
│       └── manifest.ts         // Sound file paths and variant mappings
```

CHAPTER 2: EVENT ENGINE

2.1 PURPOSE

The Event Engine is the **heartbeat** of the game. It:

1. Receives raw events (ball results, match state changes)
2. Queues them in priority order
3. Broadcasts each event to ALL other engines simultaneously
4. **ENFORCES timing** — no event starts before the previous one's mandatory duration expires
5. Prevents the "everything happens too fast" problem

2.2 TYPES

```
// engines/types.ts

export type EventType =
  // Match lifecycle
  | 'MATCH_INIT'           // Match data loaded, ready to start
  | 'MATCH_START'          // First ball about to be bowled
  | 'MATCH_END'            // Final result determined
  | 'MATCH_ABANDONED'      // Match cancelled/disconnected

  // Toss
  | 'TOSS_BEGIN'           // Toss screen shown
  | 'TOSS_COIN_FLIP'       // Coin is in the air
  | 'TOSS_RESULT'          // Coin landed, winner determined
  | 'TOSS_DECISION'        // Bat/bowl choice made

  // Innings
  | 'INNINGS_START'        // An innings begins (1st or 2nd)
  | 'INNINGS_END'          // An innings concludes
  | 'INNINGS_BREAK'        // Break card shown between innings

  // Overs
  | 'OVER_START'           // New over begins
  | 'OVER_END'             // Over completed (6 legal deliveries)
  | 'POWERPLAY_START'      // Overs 1-6 begin (visual/audio cue)
  | 'POWERPLAY_END'        // Powerplay ends
  | 'MIDDLE_OVERS_START'   // Overs 7-15
  | 'DEATH_OVERS_START'    // Overs 16-20

  // Ball-by-ball
  | 'BALL_BOWLER_RUNUP'    // Bowler starts running in (animation trigger)
  | 'BALL_PICKS_READY'     // Both players have submitted their picks
  | 'BALL_RESULT'          // Result calculated and ready to display
  | 'BALL_ANIMATION_DONE'  // All animations for this ball have completed

  // Scoring events (sub-types of BALL_RESULT, fired AFTER it)
  | 'DOT_BALL'             // 0 runs, no wicket
  | 'RUNS_SCORED'          // 1, 2, or 3 runs
  | 'BOUNDARY_FOUR'        // 4 runs (boundary)
```

```
  | 'BOUNDARY_SIX'         // 6 runs (over the rope)
  | 'WIDE_BALL'            // Extra — wide
  | 'NO_BALL'              // Extra — no ball

  // Wicket events
  | 'WICKET_BOWLED'        // Stumps hit
  | 'WICKET_CAUGHT'        // Caught by fielder
  | 'WICKET_CAUGHT_BEHIND' // Caught by keeper
  | 'WICKET_LBW'           // Leg before wicket
  | 'WICKET_RUN_OUT'       // Run out
  | 'WICKET_STUMPED'       // Stumped by keeper
  | 'WICKET_HIT_WICKET'    // Batsman hit own stumps

  // Player events
  | 'NEW_BATSMAN'          // New batsman walks in after wicket
  | 'BOWLER_CHANGE'        // Bowler changed at over end
  | 'STRIKER_CHANGE'       // Strike rotated (odd runs)

  // Milestone events
  | 'MILESTONE_50'         // Batsman reaches 50
  | 'MILESTONE_100'        // Batsman reaches 100
  | 'MILESTONE_5_WICKETS'  // Bowler gets 5 wickets
  | 'HATTRICK'             // 3 wickets in 3 consecutive balls

  // Review
  | 'DRS_REVIEW_START'     // DRS review initiated
  | 'DRS_REVIEW_RESULT'    // DRS result (upheld/overturned)

  // UI-specific
  | 'UI_SHOW_CARD'         // Show a popup card (over break, wicket detail, etc.)
  | 'UI_HIDE_CARD'         // Dismiss a popup card
  | 'UI_UPDATE_SCORE'      // Scoreboard number change
  | 'UI_FLASH_BOUNDARY'    // FOUR!/SIX! flash graphic
  | 'UI_SCREEN_SHAKE'      // Camera shake effect

  // Crowd/atmosphere
  | 'CROWD_REACT'          // Crowd mood change
  | 'CROWD_CHANT_START'    // Crowd begins a chant
  | 'CROWD_MEXICAN_WAVE'   // Mexican wave triggered

  // Weather
  | 'WEATHER_CHANGE'       // Weather state changes mid-match
  | 'WEATHER_RAIN_START'   // Rain begins
  | 'WEATHER_RAIN_STOP'    // Rain stops

  // Commentary
  | 'COMMENTARY_LINE'      // A commentary line should be spoken
  | 'COMMENTARY_DUO'       // A duo commentary exchange (main + color)

  // Sound
  | 'SOUND_EFFECT'         // Play a one-shot sound effect
  | 'SOUND_AMBIENT_CHANGE' // Change ambient sound loop
  | 'SOUND_MUSIC_CHANGE';  // Change background music

export interface GameEvent {
  id: string;                    // Unique ID (uuid)
  type: EventType;               // What happened
  payload: Record<string, any>;  // Event-specific data
  timestamp: number;             // When it was created (Date.now())
  duration: number;              // How long this event's effects last (ms)
  blocking: boolean;             // If true, queue pauses until duration expires
  priority: number;              // Higher number = processed first if queued simultaneously
  source: 'local' | 'server' | 'sync'; // Where the event originated
}
```

```
export interface MatchContext {
  innings: 1 | 2;
  over: number;                  // Current over number (0-indexed)
  ball: number;                  // Ball in current over (0-5)
  phase: 'powerplay' | 'middle' | 'death';
  battingTeam: string;
  bowlingTeam: string;
  score: number;
  wickets: number;
  target: number | null;         // Only in 2nd innings
  requiredRunRate: number | null;
  currentRunRate: number;
  lastFewBalls: string[];        // Last 6 ball results for context ('dot','1','4','6','W')
  matchSituation: 'comfortable' | 'tight' | 'tense' | 'critical'; // Derived from context
  isLastOver: boolean;
  isMatchPoint: boolean;         // One wicket/boundary could end the match
}
```

2.3 IMPLEMENTATION

```
// engines/EventEngine.ts

import { GameEvent, EventType } from './types';
import { v4 as uuid } from 'uuid'; // or crypto.randomUUID()

type EventListener = (payload: Record<string, any>, event: GameEvent) => void | Promise<void>;

export class EventEngine {
  private queue: GameEvent[] = [];
  private listeners: Map<EventType, EventListener[]> = new Map();
  private wildcardListeners: EventListener[] = []; // Listen to ALL events
  private isProcessing: boolean = false;
  private currentEvent: GameEvent | null = null;
  private isPaused: boolean = false;
  private eventLog: GameEvent[] = []; // Full history for replay
  private onEventProcessed?: (event: GameEvent) => void; // Callback for debugging

  // ── REGISTRATION ──

  /**
   * Register a listener for a specific event type.
   * Multiple listeners can be registered for the same type.
   * They all fire simultaneously via Promise.all.
   */
  on(type: EventType, callback: EventListener): () => void {
    if (!this.listeners.has(type)) {
      this.listeners.set(type, []);
    }
    this.listeners.get(type)!.push(callback);

    // Return unsubscribe function
    return () => {
      const arr = this.listeners.get(type);
      if (arr) {
        const idx = arr.indexOf(callback);
        if (idx !== -1) arr.splice(idx, 1);
      }
    };
  }

  /**
   * Register a listener that fires on EVERY event.
   * Useful for logging, analytics, and the replay system.
```

```
 */
onAny(callback: EventListener): () => void {
  this.wildcardListeners.push(callback);
  return () => {
    const idx = this.wildcardListeners.indexOf(callback);
    if (idx !== -1) this.wildcardListeners.splice(idx, 1);
  };
}

// ── EVENT EMISSION ──

/**
 * Add an event to the queue. If nothing is currently processing,
 * immediately start processing.
 */
emit(
  type: EventType,
  payload: Record<string, any> = {},
  options: Partial<Pick<GameEvent, 'duration' | 'blocking' | 'priority' | 'source'>> = {}
): string {
  const event: GameEvent = {
    id: uuid(),
    type,
    payload,
    timestamp: Date.now(),
    duration: options.duration ?? this.getDefaultDuration(type),
    blocking: options.blocking ?? this.isBlockingByDefault(type),
    priority: options.priority ?? this.getDefaultPriority(type),
    source: options.source ?? 'local',
  };

  this.queue.push(event);
  // Sort by priority (highest first), then by timestamp (oldest first for same priority)
  this.queue.sort((a, b) => {
    if (b.priority !== a.priority) return b.priority - a.priority;
    return a.timestamp - b.timestamp;
  });

  if (!this.isProcessing && !this.isPaused) {
    this.processNext();
  }

  return event.id;
}

/**
 * Emit multiple events in sequence (they'll be queued in order).
 * Useful for complex sequences like wicket → walk off → new batsman.
 */
emitSequence(events: Array<{ type: EventType; payload?: Record<string, any>; duration?: number }>): void {
  events.forEach((e, index) => {
    this.emit(e.type, e.payload ?? {}, {
      duration: e.duration,
      priority: 100 - index, // First event has highest priority
    });
  });
}

// ── PROCESSING ──

private async processNext(): Promise<void> {
  if (this.queue.length === 0 || this.isPaused) {
    this.isProcessing = false;
    return;
  }
```

```
  this.isProcessing = true;
  this.currentEvent = this.queue.shift()!;
  this.eventLog.push(this.currentEvent);

  // Notify ALL listeners for this event type simultaneously
  const typeListeners = this.listeners.get(this.currentEvent.type) || [];
  const allListeners = [...typeListeners, ...this.wildcardListeners];

  try {
    await Promise.all(
      allListeners.map(fn => fn(this.currentEvent!.payload, this.currentEvent!))
    );
  } catch (error) {
    console.error(`Error processing event ${this.currentEvent.type}:`, error);
  }

  // If this event is blocking, wait for its duration before proceeding
  if (this.currentEvent.blocking && this.currentEvent.duration > 0) {
    await this.sleep(this.currentEvent.duration);
  }

  // Callback for debugging/monitoring
  this.onEventProcessed?.(this.currentEvent);

  this.currentEvent = null;
  this.processNext();
}

// ── CONTROL ──

pause(): void {
  this.isPaused = true;
}

resume(): void {
  this.isPaused = false;
  if (!this.isProcessing && this.queue.length > 0) {
    this.processNext();
  }
}

clearQueue(): void {
  this.queue = [];
}

getEventLog(): GameEvent[] {
  return [...this.eventLog];
}

isCurrentlyBlocking(): boolean {
  return this.isProcessing && (this.currentEvent?.blocking ?? false);
}

// ── DEFAULT CONFIGURATIONS ──

private getDefaultDuration(type: EventType): number {
  const durations: Partial<Record<EventType, number>> = {
    // Match lifecycle
    MATCH_START: 500,
    MATCH_END: 1000,

    // Toss
    TOSS_BEGIN: 500,
    TOSS_COIN_FLIP: 2500,
```

```
    TOSS_RESULT: 1500,
    TOSS_DECISION: 1500,

    // Innings
    INNINGS_START: 1000,
    INNINGS_END: 1000,
    INNINGS_BREAK: 3000,  // Minimum 3s before player can skip

    // Ball sequence
    BALL_BOWLER_RUNUP: 600,
    BALL_RESULT: 200,     // Brief — sub-events carry the real duration

    // Scoring — THESE ARE THE CRITICAL TIMINGS
    DOT_BALL: 900,
    RUNS_SCORED: 1300,
    BOUNDARY_FOUR: 2800,
    BOUNDARY_SIX: 3800,

    // Wickets — THE LONGEST EVENTS (must breathe)
    WICKET_BOWLED: 4200,
    WICKET_CAUGHT: 4500,
    WICKET_CAUGHT_BEHIND: 4000,
    WICKET_LBW: 5000,     // Extra time for umpire finger raise drama
    WICKET_RUN_OUT: 4500,
    WICKET_STUMPED: 3800,
    WICKET_HIT_WICKET: 3500,

    // Player transitions
    NEW_BATSMAN: 2000,
    BOWLER_CHANGE: 1500,

    // Overs
    OVER_START: 300,
    OVER_END: 2000,       // Over summary card display time

    // Milestones
    MILESTONE_50: 3000,
    MILESTONE_100: 4000,
    MILESTONE_5_WICKETS: 3500,
    HATTRICK: 5000,

    // DRS
    DRS_REVIEW_START: 1000,
    DRS_REVIEW_RESULT: 3000,

    // UI
    UI_SHOW_CARD: 200,
    UI_HIDE_CARD: 200,
    UI_UPDATE_SCORE: 400,
    UI_FLASH_BOUNDARY: 1500,
    UI_SCREEN_SHAKE: 300,

    // Phase changes
    POWERPLAY_START: 1000,
    POWERPLAY_END: 800,
    DEATH_OVERS_START: 1000,

    // Sound/Commentary (non-blocking by default)
    COMMENTARY_LINE: 0,
    COMMENTARY_DUO: 0,
    SOUND_EFFECT: 0,
    SOUND_AMBIENT_CHANGE: 0,
  };
  return durations[type] ?? 0;
}
```

```
  private isBlockingByDefault(type: EventType): boolean {
    // These events MUST complete before the next event fires
    const blockingTypes: EventType[] = [
      'TOSS_COIN_FLIP', 'TOSS_RESULT', 'TOSS_DECISION',
      'INNINGS_START', 'INNINGS_END', 'INNINGS_BREAK',
      'BALL_BOWLER_RUNUP',
      'DOT_BALL', 'RUNS_SCORED', 'BOUNDARY_FOUR', 'BOUNDARY_SIX',
      'WICKET_BOWLED', 'WICKET_CAUGHT', 'WICKET_CAUGHT_BEHIND',
      'WICKET_LBW', 'WICKET_RUN_OUT', 'WICKET_STUMPED', 'WICKET_HIT_WICKET',
      'NEW_BATSMAN', 'BOWLER_CHANGE',
      'OVER_END',
      'MILESTONE_50', 'MILESTONE_100', 'HATTRICK',
      'DRS_REVIEW_START', 'DRS_REVIEW_RESULT',
      'MATCH_END',
    ];
    return blockingTypes.includes(type);
  }

  private getDefaultPriority(type: EventType): number {
    // Higher = processed first when multiple events are queued simultaneously
    const priorities: Partial<Record<EventType, number>> = {
      MATCH_END: 100,
      WICKET_BOWLED: 90, WICKET_CAUGHT: 90, WICKET_LBW: 90,
      WICKET_RUN_OUT: 90, WICKET_STUMPED: 90, WICKET_CAUGHT_BEHIND: 90,
      BOUNDARY_SIX: 80,
      BOUNDARY_FOUR: 70,
      HATTRICK: 95,
      MILESTONE_100: 85,
      MILESTONE_50: 75,
      UI_SCREEN_SHAKE: 60,
      SOUND_EFFECT: 50,
      COMMENTARY_LINE: 40,
      CROWD_REACT: 30,
      UI_UPDATE_SCORE: 20,
    };
    return priorities[type] ?? 10;
  }

  private sleep(ms: number): Promise<void> {
    return new Promise(resolve => setTimeout(resolve, ms));
  }
}
```

2.4 EVENT FLOW — COMPLETE BALL SEQUENCE

Here is the EXACT sequence of events fired for a single ball delivery, with timings:

```
Timeline (ms)   Event                      Duration  Blocking  What Happens
═══════════     ═══════════════════        ═══════   ═══════   ═══════════════════
0               BALL_BOWLER_RUNUP          600       YES       Bowler runs in animation
                                                               Crowd murmur builds slightly
                                                               Hand selector buttons ENABLED
                                                               "The bowler runs in..." commentary (maybe)

600             (Player picks a number — hand selector button tapped)
                (Both PvP players' picks sent to server via SyncEngine)
                (Server resolves result via GameplayEngine)

~800            BALL_PICKS_READY           0         NO        Both picks received
                                                               Hand selector buttons DISABLED
                                                               Brief suspense pause (200ms)

~1000           BALL_RESULT                200       YES       Result determined
```

```
                                                        Dispatches sub-event below:

~1200           [ONE OF THE FOLLOWING]:

                IF DOT_BALL:               900       YES
                  → SoundEngine: ball into gloves thud
                  → AnimationEngine: defensive prod, ball to keeper
                  → UIEngine: grey dot on over tracker (pop animation)
                  → CommentaryEngine: "Defended. Dot ball." (50% chance of speaking)
                  → CrowdEngine: mood -= 2
                  → UIEngine: score unchanged

                IF RUNS_SCORED (1/2/3):    1300      YES
                  → SoundEngine: bat tap + running footsteps
                  → AnimationEngine: push to gap + batsmen running
                  → UIEngine: green "+X" popup floats up, score rolls
                  → Over tracker: green dot
                  → CommentaryEngine: "Pushed into the gap, quick single."
                  → CrowdEngine: mood += runs × 3
                  → If odd runs: STRIKER_CHANGE fired (non-blocking)

                IF BOUNDARY_FOUR:          2800      YES
                  0ms:    SoundEngine: CRACK! (loud bat hit)
                  200ms:  AnimationEngine: drive shot, ball to boundary
                  400ms:  LightingEngine: boundary rope glow (team color)
                  600ms:  SoundEngine: crowd cheer builds
                  800ms:  UIEngine: "FOUR!" flash graphic flies in
                  800ms:  UIEngine: score counter rolls +4 (team accent flash)
                  800ms:  Over tracker: team-accent dot (pop animation)
                  1000ms: CommentaryEngine: "FOUR! Beautiful shot!"
                  1200ms: CrowdEngine: mood += 20 (some standing)
                  2800ms: UIEngine: "FOUR!" graphic fades out

                IF BOUNDARY_SIX:           3800      YES
                  0ms:    SoundEngine: MASSIVE CRACK + shockwave whoosh
                  100ms:  UIEngine: screen shake (3 jolts, ±4px, 150ms total)
                  150ms:  AnimationEngine: big hit, ball trajectory upward
                  300ms:  LightingEngine: screen flash white (100ms)
                  500ms:  SoundEngine: ball flight whoosh
                  800ms:  AnimationEngine: ball enters stands, spectator catches
                  1000ms: UIEngine: "SIX!" flash CRASHES in (scale 0→1.5→1)
                  1000ms: UIEngine: score counter rolls +6 (red glow + particles)
                  1000ms: Over tracker: red dot (pop animation)
                  1200ms: SoundEngine: crowd ERUPTION (maximum volume)
                  1200ms: LightingEngine: floodlight wave sequence (600ms)
                  1500ms: LightingEngine: firework bursts (3-5 bursts, team-colored)
                  1500ms: CommentaryEngine: "SIX! MAXIMUM!" (excited, rate 1.3)
                  1800ms: CrowdEngine: mood = 100 (pandemonium)
                  2000ms: CommentaryEngine: color commentator follow-up (1.5s after main)
                  3000ms: UIEngine: "SIX!" graphic fades, fireworks end
                  3800ms: Everything settles, ready for next ball

                IF WICKET_*:               4000-5000 YES
                  0ms:    TIME FREEZE — everything pauses (300ms visual freeze)
                  300ms:  Dismissal-specific animation begins:
                          BOWLED: Ball hits stumps → stumps EXPLODE
                          CAUGHT: Edge → ball arcs → fielder DIVES → catches
                          LBW: Ball hits pad → appeal → umpire finger RAISES (slow)
                          RUN OUT: Running → throw → DIRECT HIT → replays
                  300ms:  SoundEngine: dismissal-specific sound
                          BOWLED: stump rattle + timber crash
                          CAUGHT: edge nick → crowd gasp → "CATCH!" shout
                          LBW: pad thud → "HOWZAT!" appeal shout
                  800ms:  LightingEngine: screen flash RED (150ms)
                  1000ms: UIEngine: "OUT!" flash graphic (red, cracked style)
```

```
                  1200ms: SoundEngine: crowd reaction (perspective-dependent — see 5.4)
                  1500ms: CommentaryEngine: dramatic pause → "BOWLED HIM!" (rate 0.9, slow)
                  2000ms: AnimationEngine: bowler celebration (team-specific)
                  2500ms: UIEngine: OUT card slides in (wicket detail: name, dismissal, score)
                  2500ms: Over tracker: skull/W dot
                  3000ms: CommentaryEngine: color commentator follow-up
                  4000ms: UIEngine: OUT card can be dismissed (CONTINUE button activates)
                  4500ms: UIEngine: dismissed batsman avatar walks off
                  4800ms: NEW_BATSMAN event fires → new batsman walks in animation
                  5000ms: Ready for next ball

~2100-6200      BALL_ANIMATION_DONE        0         NO        Cleanup, enable hand selector
                                                                Check: is the over complete? (ball 6 of 6?)
                                                                Check: is the innings over? (all out / overs done?)
                                                                Check: is the match over? (target reached / innings 2 c

                IF over complete:
                  → OVER_END fires (see below)
                IF innings complete:
                  → INNINGS_END fires (see below)
                IF match complete:
                  → MATCH_END fires (see below)
                ELSE:
                  → OVER_START or next BALL_BOWLER_RUNUP fires
```

2.5 OVER END SEQUENCE

```
OVER_END (blocking: true, duration: 2000ms + user interaction)

  0ms:     UIEngine: Over Break card slides up from bottom (spring, 300ms)
           Card contains: ball-by-ball dots, runs in over, bowler figures, batsman scores, match situation
  300ms:   CommentaryEngine: "End of over X. [Runs] off the over."
  500ms:   UIEngine: If T10/T20, "NEXT BOWLER" button appears in the card
  2000ms:  UIEngine: "CONTINUE ▶" button activates (or auto-dismiss after 5s)

  On CONTINUE tap:
    → UI_HIDE_CARD fires (card slides down, 200ms)
    → If bowler change needed: BOWLER_CHANGE fires
    → If strike changes: STRIKER_CHANGE fires
    → OVER_START fires for next over
    → BALL_BOWLER_RUNUP fires for first ball of new over
```

2.6 INNINGS CHANGE SEQUENCE

```
INNINGS_END (blocking: true, duration: 1000ms)
  → Innings summary calculated
  → All batting/bowling stats finalized

INNINGS_BREAK (blocking: true, duration: 3000ms + user interaction)
  0ms:     Full-screen Innings Break card appears
           Shows: team score, top scorer, best bowler, boundaries, run rate
           TARGET prominently displayed for 2nd innings
           "START 2ND INNINGS" button is GREYED OUT for 3 seconds
  3000ms:  Button activates (turns green, becomes tappable)
  On tap:
    → Card dismisses
    → SyncEngine: sync point (both PvP players must be ready)
    → Teams swap (visual animation of players crossing)
    → INNINGS_START fires for innings 2
    → First BALL_BOWLER_RUNUP of innings 2
```

2.7 MATCH END SEQUENCE

```
MATCH_END (blocking: true, duration: 1000ms)
  0ms:     Final ball result animation completes
  500ms:   Brief pause (the moment of realization)
  1000ms:  Result determined: WIN / LOSS / DRAW

  IF WIN:
    → LightingEngine: full brightness + team light wash
    → SoundEngine: victory fanfare (5s orchestral sting)
    → UIEngine: transition to Match Result screen (Win state)
    → CrowdEngine: continuous celebration (flags, cheering, fireworks)
    → CommentaryEngine: "[Team] WIN! What a match!"

  IF LOSS:
    → LightingEngine: dim to 40% + grey wash
    → SoundEngine: defeat sting (3s somber)
    → UIEngine: transition to Match Result screen (Loss state)
    → CrowdEngine: quiet, subdued
    → CommentaryEngine: "And that's the end. [Team] fall short."
```

CHAPTER 3: SYNC ENGINE

3.1 PURPOSE

Ensures both PvP players experience every event at the SAME time. Prevents: one player seeing the toss result before the other, one player's match loading faster, ball results appearing at different times, innings change happening for one player before the other.

3.2 IMPLEMENTATION

```
// engines/SyncEngine.ts

import { RealtimeChannel } from '@supabase/supabase-js';
import { supabase } from '@/lib/supabase';

interface SyncPoint {
  id: string;
  resolve: () => void;
  timeout: NodeJS.Timeout;
}

export class SyncEngine {
  private channel: RealtimeChannel | null = null;
  private matchId: string = '';
  private userId: string = '';
  private opponentId: string = '';
  private pendingSyncPoints: Map<string, SyncPoint> = new Map();
  private ballPicks: Map<string, { pick: number; role: 'batting' | 'bowling' }> = new Map();
  private onOpponentPick?: (pick: number) => void;
  private onBallResult?: (result: BallResult) => void;
  private onOpponentReady?: (pointId: string) => void;
  private isHost: boolean = false; // First player to join is host (resolves ties)
```

```
// ── CONNECTION ──

async connect(matchId: string, userId: string, opponentId: string): Promise<void> {
  this.matchId = matchId;
  this.userId = userId;
  this.opponentId = opponentId;

  this.channel = supabase.channel(`match-${matchId}`, {
    config: { broadcast: { ack: true } } // Ensure delivery
  });

  this.channel
    .on('broadcast', { event: 'sync_ready' }, ({ payload }) => {
      this.handleSyncReady(payload);
    })
    .on('broadcast', { event: 'ball_pick' }, ({ payload }) => {
      this.handleBallPick(payload);
    })
    .on('broadcast', { event: 'ball_result' }, ({ payload }) => {
      this.handleBallResult(payload);
    })
    .on('broadcast', { event: 'disconnect' }, ({ payload }) => {
      this.handleDisconnect(payload);
    });

  await this.channel.subscribe();
}

disconnect(): void {
  if (this.channel) {
    this.channel.send({
      type: 'broadcast',
      event: 'disconnect',
      payload: { userId: this.userId }
    });
    this.channel.unsubscribe();
    this.channel = null;
  }
}

// ── SYNC POINTS ──

/**
 * Both players must reach this sync point before either can proceed.
 * Used at: toss start, toss result, innings start, innings break, match end
 */
async waitForSync(pointId: string, timeoutMs: number = 15000): Promise<void> {
  // Tell opponent "I'm ready at this point"
  await this.channel?.send({
    type: 'broadcast',
    event: 'sync_ready',
    payload: { pointId, userId: this.userId, timestamp: Date.now() }
  });

  // Check if opponent already signaled ready for this point
  if (this.pendingSyncPoints.has(`opponent_${pointId}`)) {
    this.pendingSyncPoints.get(`opponent_${pointId}`)!.resolve();
    this.pendingSyncPoints.delete(`opponent_${pointId}`);
    return;
  }

  // Wait for opponent to also reach this point
  return new Promise<void>((resolve) => {
    const timeout = setTimeout(() => {
      console.warn(`Sync timeout at ${pointId} — proceeding anyway`);
```

```
      this.pendingSyncPoints.delete(pointId);
      resolve();
    }, timeoutMs);

    this.pendingSyncPoints.set(pointId, { id: pointId, resolve: () => {
      clearTimeout(timeout);
      this.pendingSyncPoints.delete(pointId);
      resolve();
    }, timeout });
  });
}

private handleSyncReady(payload: { pointId: string; userId: string }): void {
  if (payload.userId === this.opponentId) {
    const pending = this.pendingSyncPoints.get(payload.pointId);
    if (pending) {
      pending.resolve();
    } else {
      // Opponent arrived first — store it so our waitForSync resolves immediately
      this.pendingSyncPoints.set(`opponent_${payload.pointId}`, {
        id: payload.pointId,
        resolve: () => {},
        timeout: setTimeout(() => {}, 0)
      });
    }
  }
}

// ── BALL PICKS ──

/**
 * Send this player's number pick for the current ball.
 * Waits for opponent's pick. Once both are in, resolves the ball server-side.
 */
async sendBallPick(pick: number, role: 'batting' | 'bowling', ballId: string): Promise<void> {
  await this.channel?.send({
    type: 'broadcast',
    event: 'ball_pick',
    payload: { userId: this.userId, pick, role, ballId, timestamp: Date.now() }
  });

  // Store own pick
  this.ballPicks.set(`${this.userId}_${ballId}`, { pick, role });
}

private async handleBallPick(payload: {
  userId: string; pick: number; role: 'batting' | 'bowling'; ballId: string
}): Promise<void> {
  if (payload.userId === this.opponentId) {
    // Store opponent's pick
    this.ballPicks.set(`${payload.userId}_${payload.ballId}`, {
      pick: payload.pick,
      role: payload.role
    });

    this.onOpponentPick?.(payload.pick);

    // Check if BOTH picks are in
    const myPick = this.ballPicks.get(`${this.userId}_${payload.ballId}`);
    const theirPick = this.ballPicks.get(`${this.opponentId}_${payload.ballId}`);

    if (myPick && theirPick) {
      // Both picks received — resolve the ball
      // Only the HOST resolves to prevent duplicate calculations
      if (this.isHost) {
```

```
        await this.resolveBall(payload.ballId, myPick, theirPick);
      }
    }
  }
}

private async resolveBall(
  ballId: string,
  myPick: { pick: number; role: string },
  theirPick: { pick: number; role: string }
): Promise<void> {
  // Determine batting and bowling picks
  const battingPick = myPick.role === 'batting' ? myPick.pick : theirPick.pick;
  const bowlingPick = myPick.role === 'bowling' ? myPick.pick : theirPick.pick;

  // Call server-side edge function for fair resolution
  const { data: result } = await supabase.functions.invoke('resolve-ball', {
    body: {
      matchId: this.matchId,
      ballId,
      battingPick,
      bowlingPick,
      // Additional context sent for stat-modified calculations
      // (batsman ID, bowler ID, pitch, phase — see GameplayEngine)
    }
  });

  // Broadcast result to BOTH players simultaneously
  await this.channel?.send({
    type: 'broadcast',
    event: 'ball_result',
    payload: { ballId, result, timestamp: Date.now() }
  });
}

private handleBallResult(payload: { ballId: string; result: BallResult }): void {
  // Both players receive this at the same time
  this.onBallResult?.(payload.result);
  // Clean up picks for this ball
  this.ballPicks.delete(`${this.userId}_${payload.ballId}`);
  this.ballPicks.delete(`${this.opponentId}_${payload.ballId}`);
}

// ── TIMEOUT HANDLING ──

/**
 * If opponent doesn't pick within 10 seconds, auto-assign random.
 * Called by the UI timer component.
 */
async forceOpponentPick(ballId: string): Promise<void> {
  const randomPick = Math.floor(Math.random() * 6) + 1;
  // Simulate opponent's pick
  this.handleBallPick({
    userId: this.opponentId,
    pick: randomPick,
    role: 'bowling', // or 'batting' depending on current state
    ballId
  });
}

// ── DISCONNECT HANDLING ──

private handleDisconnect(payload: { userId: string }): void {
  if (payload.userId === this.opponentId) {
    // Opponent disconnected — switch to AI mode for remaining match
```

```
      // Fire event so UI can show "Opponent disconnected. Playing vs AI."
      console.warn('Opponent disconnected');
    }
  }

  // ── SETTERS ──

  setOnOpponentPick(cb: (pick: number) => void): void { this.onOpponentPick = cb; }
  setOnBallResult(cb: (result: BallResult) => void): void { this.onBallResult = cb; }
  setIsHost(isHost: boolean): void { this.isHost = isHost; }
}
```

3.3 SYNC POINT MAP

Sync Point	When	Timeout	On Timeout
`pre_match`	Before VS screen shows	30s	Cancel match, return to lobby
`toss_start`	Before toss	15s	Auto-proceed, random toss
`toss_call_{ballId}`	Both players pick heads/tails	10s	Auto-random for absent player
`toss_decision`	Bat/bowl choice	10s	Auto-random
`vs_screen_done`	VS screen auto-timer (3s)	5s	Proceed
`innings_1_start`	Before first ball	10s	Proceed
`ball_{over}_{ball}`	Both players submit picks	10s	Auto-random for absent player
`over_end_{over}`	Over summary dismissed	10s	Auto-dismiss
`innings_break`	Both tap “Start 2nd Innings”	30s	Auto-proceed
`innings_2_start`	Before first ball of innings 2	10s	Proceed
`match_end`	Both see result	10s	Proceed

3.4 PVP TIMER — THE 10-SECOND PICK TIMER

Visual: A circular progress ring around the center of the screen (or around the VS area). The ring starts full (green) and depletes clockwise over 10 seconds.

```
10s-8s:   Ring is green, full to 80%
8s-5s:    Ring turns yellow, depleting
5s-3s:    Ring turns orange, depleting faster visually
3s-0s:    Ring turns RED, pulsing, warning beep sounds each second
0s:       Ring empty → auto-random pick assigned → ball proceeds
```

Sound: At 3s, 2s, 1s remaining: a short “tick” sound plays. At 0s: a “bzzz” error sound.

CHAPTER 4: GAMEPLAY ENGINE

4.1 PURPOSE

The math brain. Determines what happens on every ball based on player picks + character stats + match context. This runs SERVER-SIDE (Supabase Edge Function) for PvP fairness, or client-side for vs AI.

4.2 BALL RESOLUTION ALGORITHM

```
// engines/GameplayEngine.ts

export interface BallResult {
  runs: number;                          // 0-6
  isWicket: boolean;
  isDot: boolean;                        // 0 runs, no wicket
  isBoundaryFour: boolean;
  isBoundarySix: boolean;
  isWide: boolean;
  isNoBall: boolean;
  dismissalType: DismissalType | null;
  battingPick: number;
  bowlingPick: number;
  commentary: string;                    // Pre-selected commentary line
  soundEffects: string[];               // Sound IDs to play
}

export type DismissalType =
  | 'bowled' | 'caught' | 'caught_behind' | 'lbw'
  | 'run_out' | 'stumped' | 'hit_wicket';

export class GameplayEngine {

  resolveBall(
    battingPick: number,
    bowlingPick: number,
    batsman: PlayerStats,
    bowler: PlayerStats,
    context: MatchContext
  ): BallResult {

    // ── STEP 1: Wide/No Ball check (5% chance each, random) ──
    const extraRoll = Math.random();
    if (extraRoll < 0.03) {
      return this.createWideResult(bowler);
    }
    if (extraRoll < 0.05) {
      return this.createNoBallResult(battingPick, batsman);
    }

    // ── STEP 2: Numbers Match → Potential Wicket ──
    if (battingPick === bowlingPick) {
      return this.resolveMatchedNumbers(batsman, bowler, context);
    }

    // ── STEP 3: Numbers Don't Match → Runs ──
    return this.resolveRuns(battingPick, batsman, bowler, context);
  }

  private resolveMatchedNumbers(
    batsman: PlayerStats,
    bowler: PlayerStats,
    context: MatchContext
  ): BallResult {
    // Base: When numbers match, it's a potential wicket
    // But high-technique batsmen can SURVIVE
```

```
  const techDefense = (100 - batsman.technique) / 100;    // Lower technique = higher out chance
  const bowlerThreat = bowler.accuracy / 100;              // Higher accuracy = more dangerous
  const pitchFactor = this.getPitchWicketMod(context);     // Pitch conditions
  const clutchFactor = this.getClutchMod(batsman, context); // Pressure situations
  const phaseFactor = this.getPhaseWicketMod(context);     // Death overs = more wickets

  // Final out probability (0.0 to 1.0)
  const outChance = techDefense * bowlerThreat * pitchFactor * clutchFactor * phaseFactor;
  // Clamp between 5% minimum and 85% maximum
  const clampedChance = Math.max(0.05, Math.min(0.85, outChance));

  if (Math.random() < clampedChance) {
    // WICKET!
    const dismissalType = this.determineDismissalType(bowler, context);
    return {
      runs: 0,
      isWicket: true,
      isDot: false,
      isBoundaryFour: false,
      isBoundarySix: false,
      isWide: false,
      isNoBall: false,
      dismissalType,
      battingPick: 0, bowlingPick: 0, // Hidden for dramatic reveal
      commentary: '',
      soundEffects: [],
    };
  } else {
    // SURVIVED! Matched numbers but technique saved them
    // This is always a dot ball (0 runs)
    return {
      runs: 0,
      isWicket: false,
      isDot: true,
      isBoundaryFour: false,
      isBoundarySix: false,
      isWide: false,
      isNoBall: false,
      dismissalType: null,
      battingPick: 0, bowlingPick: 0,
      commentary: '',
      soundEffects: [],
    };
  }
}

private resolveRuns(
  battingPick: number,
  batsman: PlayerStats,
  bowler: PlayerStats,
  context: MatchContext
): BallResult {
  // Base runs = what the batsman picked
  let runs = battingPick;

  // ── Modifiers ──
  const powerMod = batsman.power / 100;                    // High power = runs hold value
  const bowlerSaveMod = (100 - bowler.accuracy) / 100;     // High accuracy bowler REDUCES runs
  const phaseMod = this.getPhaseBattingMod(context, batsman);
  const pitchMod = this.getPitchBattingMod(context);

  // Apply modifiers
  let modifiedRuns = runs * powerMod * bowlerSaveMod * phaseMod * pitchMod;
```

```
  // Special abilities can modify here
  modifiedRuns = this.applySpecialAbilities(modifiedRuns, batsman, bowler, context);

  // Round and clamp
  runs = Math.round(modifiedRuns);
  runs = Math.max(0, Math.min(6, runs));

  // Determine if it's a boundary
  const isFour = runs === 4;
  const isSix = runs === 6;

  return {
    runs,
    isWicket: false,
    isDot: runs === 0,
    isBoundaryFour: isFour,
    isBoundarySix: isSix,
    isWide: false,
    isNoBall: false,
    dismissalType: null,
    battingPick, bowlingPick: 0,
    commentary: '',
    soundEffects: [],
  };
}

// ── DISMISSAL TYPE DETERMINATION ──

private determineDismissalType(bowler: PlayerStats, context: MatchContext): DismissalType {
  const roll = Math.random();
  const isSpin = bowler.bowlingType?.includes('spin');
  const isPace = bowler.bowlingType?.includes('fast') || bowler.bowlingType?.includes('med');

  if (isPace) {
    if (roll < 0.30) return 'bowled';         // 30%
    if (roll < 0.55) return 'caught';          // 25%
    if (roll < 0.70) return 'caught_behind';   // 15%
    if (roll < 0.85) return 'lbw';             // 15%
    if (roll < 0.95) return 'run_out';          // 10%
    return 'hit_wicket';                        // 5%
  } else if (isSpin) {
    if (roll < 0.20) return 'bowled';          // 20%
    if (roll < 0.45) return 'caught';          // 25%
    if (roll < 0.55) return 'caught_behind';   // 10%
    if (roll < 0.75) return 'lbw';             // 20%
    if (roll < 0.85) return 'stumped';          // 10%
    if (roll < 0.95) return 'run_out';          // 10%
    return 'hit_wicket';                        // 5%
  }
  return 'caught'; // Default fallback
}

// ── MODIFIER FUNCTIONS ──

private getPhaseBattingMod(context: MatchContext, batsman: PlayerStats): number {
  switch (context.phase) {
    case 'powerplay':
      // Powerplay: slight batting advantage, fielding restrictions help
      return batsman.specialAbility === 'Explosive Opener' ? 1.35 : 1.12;
    case 'middle':
      return 1.0; // Balanced
    case 'death':
      // Death overs: power hitters thrive
      return batsman.power > 80 ? 1.28 : 1.10;
    default: return 1.0;
```

```
  }
}

private getPhaseWicketMod(context: MatchContext): number {
  switch (context.phase) {
    case 'powerplay': return 1.1;    // New ball = slightly more wickets
    case 'middle': return 0.9;       // Set batsmen, fewer wickets
    case 'death': return 1.15;       // High risk = more wickets
    default: return 1.0;
  }
}

private getPitchBattingMod(context: MatchContext): number {
  const pitch = context.pitch || 'flat';
  switch (pitch) {
    case 'green_top': return 0.85;     // Seam movement hurts batsmen
    case 'dustbowl': return 0.90;      // Spin helps, batting harder
    case 'flat': return 1.15;          // Batting paradise
    case 'minefield': return 0.80;     // Variable bounce, chaos
    default: return 1.0;
  }
}

private getPitchWicketMod(context: MatchContext): number {
  const pitch = context.pitch || 'flat';
  switch (pitch) {
    case 'green_top': return 1.20;
    case 'dustbowl': return 1.15;
    case 'flat': return 0.85;
    case 'minefield': return 1.30;
    default: return 1.0;
  }
}

private getClutchMod(batsman: PlayerStats, context: MatchContext): number {
  // High clutch stat = HARDER to get out in pressure situations
  // Low clutch = MORE likely to get out under pressure
  if (context.matchSituation === 'critical' || context.matchSituation === 'tense') {
    // Clutch stat reduces out chance in pressure
    return (100 - batsman.clutch * 0.5) / 100;
    // Clutch 90: modifier = 0.55 (45% reduction in out chance — clutch player!)
    // Clutch 30: modifier = 0.85 (only 15% reduction — chokes under pressure)
  }
  return 1.0; // No pressure = no modifier
}

private applySpecialAbilities(
  runs: number,
  batsman: PlayerStats,
  bowler: PlayerStats,
  context: MatchContext
): number {
  // Apply batsman special abilities
  switch (batsman.specialAbilityId) {
    case 'helicopter_shot':
      // 20% chance any 4 becomes a 6
      if (Math.round(runs) === 4 && Math.random() < 0.20) return 6;
      break;
    case 'hitman_pull':
      // 30% chance picks of 5 or 6 become 6
      if (runs >= 5 && Math.random() < 0.30) return 6;
      break;
    case '360_player':
      // +20% power always
      runs *= 1.20;
```

```
        break;
      case 'king_kohli':
        // In chases (2nd innings), all modifiers +15%
        if (context.innings === 2) runs *= 1.15;
        break;
      case 'explosive_opener':
        // First 3 overs, power is 99 equivalent
        if (context.over < 3) runs *= 1.30;
        break;
      case 'six_machine':
        // Every 4 has 35% chance to become 6
        if (Math.round(runs) === 4 && Math.random() < 0.35) return 6;
        break;
      case 'finisher':
        // +20% power when < 5 overs remaining
        if (context.oversRemaining && context.oversRemaining < 5) runs *= 1.20;
        break;
      case 'rinku_5_sixes':
        // In last over, power becomes 99
        if (context.isLastOver) runs *= 1.40;
        break;
    }
    return runs;
  }

  // ── MATCHMAKING RATING ──

  /**
   * Fair matchmaking based on per-match AVERAGES, not cumulative totals.
   * A player with 100 matches and one with 10 matches are compared fairly.
   */
  calculateMatchmakingRating(profile: UserProfile): number {
    const matchesPlayed = Math.max(profile.matchesPlayed, 1); // Avoid division by zero

    const avgRuns = profile.totalRunsScored / matchesPlayed;
    const avgWickets = profile.totalWicketsTaken / matchesPlayed;
    const winRate = profile.matchesWon / matchesPlayed;

    // Weighted formula
    const rating =
      (avgRuns * 2) +              // Run scoring ability
      (avgWickets * 15) +          // Bowling ability (weighted higher — wickets are harder)
      (winRate * 60) +             // Win percentage (most important)
      (profile.trophies * 0.1);    // Trophy count (small factor)

    return Math.round(rating);
  }

  /**
   * Two players are a fair match if their ratings are within 15% of each other.
   */
  isFairMatch(ratingA: number, ratingB: number): boolean {
    const diff = Math.abs(ratingA - ratingB);
    const avg = (ratingA + ratingB) / 2;
    return diff / avg < 0.15;
  }
}
```

4.3 MATCH SITUATION CALCULATOR

The `matchSituation` field in MatchContext is derived automatically:

```
function calculateMatchSituation(context: MatchContext): 'comfortable' | 'tight' | 'tense' | 'critical' {
  if (context.innings === 1) {
```

```
    // First innings: situation based on run rate and wickets
    if (context.wickets >= 7) return 'critical';
    if (context.wickets >= 5 && context.currentRunRate < 6) return 'tense';
    if (context.currentRunRate > 10) return 'comfortable';
    return 'tight';
  }

  // Second innings: situation based on required rate and wickets remaining
  const wicketsRemaining = 10 - context.wickets;
  const runsNeeded = (context.target ?? 0) - context.score;
  const ballsRemaining = context.totalBalls - context.ballsBowled;
  const reqRate = (runsNeeded / ballsRemaining) * 6;

  if (runsNeeded <= 0) return 'comfortable'; // Already won
  if (wicketsRemaining <= 2 && runsNeeded > 20) return 'critical';
  if (reqRate > 15 || (wicketsRemaining <= 3 && runsNeeded > 30)) return 'critical';
  if (reqRate > 10 || runsNeeded > wicketsRemaining * 15) return 'tense';
  if (reqRate < 6 && wicketsRemaining > 5) return 'comfortable';
  return 'tight';
}
```

This drives:

- Commentary tone (more dramatic in 'tense'/'critical')
- Crowd mood (higher baseline in tense situations)
- Sound ambience (heartbeat sounds in critical moments)
- Visual effects (vignette darkening in tense moments)

CHAPTER 5: SOUND ENGINE

5.1 PURPOSE

Manages ALL audio in the game: ambient loops, one-shot effects, music, and integration with the Commentary Engine. The key innovation is **perspective-aware audio** — the same event sounds different depending on whether it's good or bad for YOU.

5.2 ARCHITECTURE

```
// engines/SoundEngine.ts

import { Howl, Howler } from 'howler';

interface SoundVariant {
  id: string;
  src: string;
  volume?: number;
}

interface SoundCategory {
  variants: SoundVariant[];
  lastPlayed: number; // Index of last played variant (for anti-repetition)
}
```

```
export class SoundEngine {
  private categories: Map<string, SoundCategory> = new Map();
  private ambientLoop: Howl | null = null;
  private musicLoop: Howl | null = null;
  private activeEffects: Howl[] = []; // Currently playing effects (for cleanup)

  // Master volume controls (0.0 to 1.0)
  private volumes = {
    master: 1.0,
    effects: 0.8,
    ambient: 0.25,
    music: 0.15,
    commentary: 1.0,
  };

  // User preferences
  private enabled = {
    sound: true,
    music: true,
    vibration: true,
  };

  constructor() {
    this.loadSoundLibrary();
  }

  // ── PLAYBACK ──

  /**
   * Play a random variant from a sound category.
   * Anti-repetition: never plays the same variant twice in a row.
   */
  playEffect(categoryId: string, volumeOverride?: number): void {
    if (!this.enabled.sound) return;

    const category = this.categories.get(categoryId);
    if (!category || category.variants.length === 0) {
      console.warn(`Sound category not found: ${categoryId}`);
      return;
    }

    // Pick a random variant that isn't the last one played
    let index: number;
    if (category.variants.length === 1) {
      index = 0;
    } else {
      do {
        index = Math.floor(Math.random() * category.variants.length);
      } while (index === category.lastPlayed);
    }

    category.lastPlayed = index;
    const variant = category.variants[index];
    const volume = (volumeOverride ?? variant.volume ?? 1.0) * this.volumes.effects * this.volumes.master;

    const howl = new Howl({
      src: [variant.src],
      volume,
      onend: () => {
        // Remove from active effects
        this.activeEffects = this.activeEffects.filter(h => h !== howl);
      }
    });

    howl.play();
```

```
  this.activeEffects.push(howl);
}

/**
 * Play a specific sound (not random variant).
 */
playSpecific(src: string, volume: number = 1.0): Howl {
  const howl = new Howl({
    src: [src],
    volume: volume * this.volumes.effects * this.volumes.master,
  });
  howl.play();
  return howl;
}

/**
 * Start an ambient loop (crossfades from current ambient).
 */
setAmbient(src: string, volume: number = 0.25, fadeDuration: number = 500): void {
  if (!this.enabled.sound) return;

  const newAmbient = new Howl({
    src: [src],
    loop: true,
    volume: 0,
  });

  newAmbient.play();
  newAmbient.fade(0, volume * this.volumes.ambient * this.volumes.master, fadeDuration);

  if (this.ambientLoop) {
    const old = this.ambientLoop;
    old.fade(old.volume(), 0, fadeDuration);
    setTimeout(() => old.unload(), fadeDuration + 100);
  }

  this.ambientLoop = newAmbient;
}

/**
 * Adjust ambient volume (e.g., CrowdEngine changes crowd volume based on mood).
 */
setAmbientVolume(volume: number, fadeDuration: number = 300): void {
  if (this.ambientLoop) {
    this.ambientLoop.fade(
      this.ambientLoop.volume(),
      volume * this.volumes.ambient * this.volumes.master,
      fadeDuration
    );
  }
}

/**
 * Stop all currently playing effects (for scene transitions).
 */
stopAllEffects(): void {
  this.activeEffects.forEach(h => h.stop());
  this.activeEffects = [];
}

// ── VOLUME CONTROL ──

setMasterVolume(v: number): void { this.volumes.master = v; Howler.volume(v); }
setEffectsVolume(v: number): void { this.volumes.effects = v; }
setAmbientVolume_master(v: number): void { this.volumes.ambient = v; }
```

```
  setMusicVolume(v: number): void { this.volumes.music = v; }
  setCommentaryVolume(v: number): void { this.volumes.commentary = v; }
  setSoundEnabled(enabled: boolean): void { this.enabled.sound = enabled; if (!enabled) this.stopAllEffects(); }
  setMusicEnabled(enabled: boolean): void { this.enabled.music = enabled; }

  // ── HAPTICS ──

  vibrate(pattern: 'light' | 'medium' | 'heavy' | 'error' | 'success'): void {
    if (!this.enabled.vibration || !navigator.vibrate) return;
    const patterns: Record<string, number | number[]> = {
      light: 10,
      medium: 25,
      heavy: 50,
      error: [50, 30, 50],
      success: [10, 20, 10],
    };
    navigator.vibrate(patterns[pattern]);
  }
}
```

5.3 SOUND LIBRARY — COMPLETE CATALOG

```
// data/sounds/manifest.ts

export const SOUND_MANIFEST = {
  // ── BAT HITS ──
  bat_hit_soft: {
    variants: [
      { id: 'bhs1', src: '/sounds/bat/hit_soft_1.mp3' },
      { id: 'bhs2', src: '/sounds/bat/hit_soft_2.mp3' },
      { id: 'bhs3', src: '/sounds/bat/hit_soft_3.mp3' },
    ],
    description: 'Gentle tap for dot balls and singles'
  },
  bat_hit_medium: {
    variants: [
      { id: 'bhm1', src: '/sounds/bat/hit_medium_1.mp3' },
      { id: 'bhm2', src: '/sounds/bat/hit_medium_2.mp3' },
      { id: 'bhm3', src: '/sounds/bat/hit_medium_3.mp3' },
      { id: 'bhm4', src: '/sounds/bat/hit_medium_4.mp3' },
    ],
    description: 'Firm hit for 2s and 3s'
  },
  bat_hit_hard: {
    variants: [
      { id: 'bhh1', src: '/sounds/bat/hit_hard_1.mp3' },
      { id: 'bhh2', src: '/sounds/bat/hit_hard_2.mp3' },
      { id: 'bhh3', src: '/sounds/bat/hit_hard_3.mp3' },
      { id: 'bhh4', src: '/sounds/bat/hit_hard_4.mp3' },
      { id: 'bhh5', src: '/sounds/bat/hit_hard_5.mp3' },
    ],
    description: 'Loud CRACK for boundaries (4s)'
  },
  bat_hit_massive: {
    variants: [
      { id: 'bhx1', src: '/sounds/bat/hit_massive_1.mp3' },
      { id: 'bhx2', src: '/sounds/bat/hit_massive_2.mp3' },
      { id: 'bhx3', src: '/sounds/bat/hit_massive_3.mp3' },
      { id: 'bhx4', src: '/sounds/bat/hit_massive_4.mp3' },
      { id: 'bhx5', src: '/sounds/bat/hit_massive_5.mp3' },
    ],
    description: 'ENORMOUS crack + shockwave for sixes'
  },
```

```
// ── BALL SOUNDS ──
ball_bounce: {
  variants: [
    { id: 'bb1', src: '/sounds/ball/bounce_1.mp3' },
    { id: 'bb2', src: '/sounds/ball/bounce_2.mp3' },
    { id: 'bb3', src: '/sounds/ball/bounce_3.mp3' },
  ],
  description: 'Ball pitching on the deck'
},
ball_into_gloves: {
  variants: [
    { id: 'big1', src: '/sounds/ball/gloves_1.mp3' },
    { id: 'big2', src: '/sounds/ball/gloves_2.mp3' },
    { id: 'big3', src: '/sounds/ball/gloves_3.mp3' },
  ],
  description: 'Ball into keeper gloves (dot ball)'
},
ball_edge: {
  variants: [
    { id: 'be1', src: '/sounds/ball/edge_1.mp3' },
    { id: 'be2', src: '/sounds/ball/edge_2.mp3' },
  ],
  description: 'Faint edge off the bat'
},
ball_pad_hit: {
  variants: [
    { id: 'bp1', src: '/sounds/ball/pad_1.mp3' },
    { id: 'bp2', src: '/sounds/ball/pad_2.mp3' },
  ],
  description: 'Ball hitting batting pad (LBW)'
},
ball_flight_whoosh: {
  variants: [
    { id: 'bfw1', src: '/sounds/ball/flight_1.mp3' },
  ],
  description: 'Ball sailing through the air (sixes)'
},

// ── STUMPS ──
stumps_hit: {
  variants: [
    { id: 'sh1', src: '/sounds/stumps/hit_1.mp3' },
    { id: 'sh2', src: '/sounds/stumps/hit_2.mp3' },
    { id: 'sh3', src: '/sounds/stumps/hit_3.mp3' },
    { id: 'sh4', src: '/sounds/stumps/hit_4.mp3' },
  ],
  description: 'Stumps shattered — timber! Various intensities'
},
bails_flying: {
  variants: [
    { id: 'bf1', src: '/sounds/stumps/bails_1.mp3' },
  ],
  description: 'Bails dislodged (lighter than stumps_hit)'
},

// ── CROWD ──
crowd_cheer_mild: {
  variants: [
    { id: 'ccm1', src: '/sounds/crowd/cheer_mild_1.mp3' },
    { id: 'ccm2', src: '/sounds/crowd/cheer_mild_2.mp3' },
    { id: 'ccm3', src: '/sounds/crowd/cheer_mild_3.mp3' },
  ],
  description: 'Polite applause for singles/doubles'
},
```

```
crowd_cheer_excited: {
  variants: [
    { id: 'cce1', src: '/sounds/crowd/cheer_excited_1.mp3' },
    { id: 'cce2', src: '/sounds/crowd/cheer_excited_2.mp3' },
    { id: 'cce3', src: '/sounds/crowd/cheer_excited_3.mp3' },
    { id: 'cce4', src: '/sounds/crowd/cheer_excited_4.mp3' },
    { id: 'cce5', src: '/sounds/crowd/cheer_excited_5.mp3' },
  ],
  description: 'Loud cheer for boundaries (FOUR)'
},
crowd_eruption: {
  variants: [
    { id: 'cer1', src: '/sounds/crowd/eruption_1.mp3' },
    { id: 'cer2', src: '/sounds/crowd/eruption_2.mp3' },
    { id: 'cer3', src: '/sounds/crowd/eruption_3.mp3' },
    { id: 'cer4', src: '/sounds/crowd/eruption_4.mp3' },
    { id: 'cer5', src: '/sounds/crowd/eruption_5.mp3' },
  ],
  description: 'MAXIMUM crowd roar for sixes — deafening'
},
crowd_gasp: {
  variants: [
    { id: 'cg1', src: '/sounds/crowd/gasp_1.mp3' },
    { id: 'cg2', src: '/sounds/crowd/gasp_2.mp3' },
    { id: 'cg3', src: '/sounds/crowd/gasp_3.mp3' },
  ],
  description: 'Collective gasp (close calls, edges, near misses)'
},
crowd_groan: {
  variants: [
    { id: 'cgr1', src: '/sounds/crowd/groan_1.mp3' },
    { id: 'cgr2', src: '/sounds/crowd/groan_2.mp3' },
    { id: 'cgr3', src: '/sounds/crowd/groan_3.mp3' },
  ],
  description: 'Disappointment/groan (opponent scores, your player out)'
},
crowd_appeal: {
  variants: [
    { id: 'ca1', src: '/sounds/crowd/appeal_1.mp3' },
    { id: 'ca2', src: '/sounds/crowd/appeal_2.mp3' },
    { id: 'ca3', src: '/sounds/crowd/appeal_3.mp3' },
    { id: 'ca4', src: '/sounds/crowd/appeal_4.mp3' },
    { id: 'ca5', src: '/sounds/crowd/appeal_5.mp3' },
    { id: 'ca6', src: '/sounds/crowd/appeal_6.mp3' },
  ],
  description: '"HOWZAT!" appeals at various intensities'
},
crowd_celebration_sustained: {
  variants: [
    { id: 'ccs1', src: '/sounds/crowd/celebration_1.mp3' },
    { id: 'ccs2', src: '/sounds/crowd/celebration_2.mp3' },
  ],
  description: 'Sustained celebration (wicket taken, match won) — 3-5 seconds'
},

// ── AMBIENT LOOPS (Continuous) ──
ambient_crowd_quiet: {
  variants: [{ id: 'acq', src: '/sounds/ambient/crowd_quiet_loop.mp3' }],
  description: 'Low murmur, between balls, quiet moments'
},
ambient_crowd_moderate: {
  variants: [{ id: 'acm', src: '/sounds/ambient/crowd_moderate_loop.mp3' }],
  description: 'Active crowd, during overs, moderate energy'
},
ambient_crowd_tense: {
```

```
  variants: [{ id: 'act', src: '/sounds/ambient/crowd_tense_loop.mp3' }],
  description: 'Quiet tension, close match, last overs'
},
ambient_crowd_loud: {
  variants: [{ id: 'acl', src: '/sounds/ambient/crowd_loud_loop.mp3' }],
  description: 'Loud sustained crowd energy, exciting phase'
},
ambient_rain: {
  variants: [{ id: 'ar', src: '/sounds/ambient/rain_loop.mp3' }],
  description: 'Rain on stadium roof'
},
ambient_wind: {
  variants: [{ id: 'aw', src: '/sounds/ambient/wind_loop.mp3' }],
  description: 'Wind for dust storm weather'
},

// ── UI SOUNDS ──
ui_button_tap: {
  variants: [
    { id: 'ubt1', src: '/sounds/ui/tap_1.mp3' },
    { id: 'ubt2', src: '/sounds/ui/tap_2.mp3' },
    { id: 'ubt3', src: '/sounds/ui/tap_3.mp3' },
  ],
  description: 'Generic button press'
},
ui_card_slide_in: {
  variants: [
    { id: 'ucsi1', src: '/sounds/ui/slide_in_1.mp3' },
    { id: 'ucsi2', src: '/sounds/ui/slide_in_2.mp3' },
  ],
  description: 'Card/panel sliding into view'
},
ui_card_slide_out: {
  variants: [
    { id: 'ucso1', src: '/sounds/ui/slide_out_1.mp3' },
  ],
  description: 'Card/panel sliding away'
},
ui_score_tick: {
  variants: [
    { id: 'ust1', src: '/sounds/ui/score_tick.mp3' },
  ],
  description: 'Scoreboard number changing (mechanical click)'
},
ui_error: {
  variants: [
    { id: 'ue1', src: '/sounds/ui/error.mp3' },
  ],
  description: 'Error/invalid action buzz'
},
ui_success: {
  variants: [
    { id: 'us1', src: '/sounds/ui/success.mp3' },
  ],
  description: 'Success confirmation ding'
},

// ── SPECIAL ──
coin_flip: {
  variants: [{ id: 'cf', src: '/sounds/special/coin_flip.mp3' }],
  description: 'Metallic coin spinning sound (toss)'
},
coin_land: {
  variants: [
    { id: 'cl1', src: '/sounds/special/coin_land_1.mp3' },
```

```
    { id: 'cl2', src: '/sounds/special/coin_land_2.mp3' },
  ],
  description: 'Coin hitting the surface and settling'
},
victory_fanfare: {
  variants: [{ id: 'vf', src: '/sounds/special/victory_fanfare.mp3' }],
  description: 'Triumphant 5-second orchestral sting'
},
defeat_sting: {
  variants: [{ id: 'ds', src: '/sounds/special/defeat_sting.mp3' }],
  description: '3-second somber sting'
},
match_start_horn: {
  variants: [{ id: 'msh', src: '/sounds/special/match_horn.mp3' }],
  description: 'Stadium horn blast to start the match'
},
firework_pop: {
  variants: [
    { id: 'fp1', src: '/sounds/special/firework_1.mp3' },
    { id: 'fp2', src: '/sounds/special/firework_2.mp3' },
    { id: 'fp3', src: '/sounds/special/firework_3.mp3' },
  ],
  description: 'Firework burst/pop (sixes, celebrations)'
},
heartbeat: {
  variants: [{ id: 'hb', src: '/sounds/special/heartbeat_loop.mp3' }],
  description: 'Heartbeat for tense moments (loops)'
},
timer_tick: {
  variants: [{ id: 'tt', src: '/sounds/special/timer_tick.mp3' }],
  description: 'Clock tick for PvP countdown timer'
},
timer_expire: {
  variants: [{ id: 'te', src: '/sounds/special/timer_expire.mp3' }],
  description: 'Buzzer when timer hits zero'
},

// ── REWARDS ──
coin_collect: {
  variants: [
    { id: 'cc1', src: '/sounds/rewards/coin_1.mp3' },
    { id: 'cc2', src: '/sounds/rewards/coin_2.mp3' },
    { id: 'cc3', src: '/sounds/rewards/coin_3.mp3' },
  ],
  description: 'Coin earned/collected clink'
},
gem_collect: {
  variants: [
    { id: 'gc1', src: '/sounds/rewards/gem_1.mp3' },
    { id: 'gc2', src: '/sounds/rewards/gem_2.mp3' },
  ],
  description: 'Gem crystal chime'
},
card_flip: {
  variants: [
    { id: 'cfl1', src: '/sounds/rewards/card_flip_1.mp3' },
    { id: 'cfl2', src: '/sounds/rewards/card_flip_2.mp3' },
  ],
  description: 'Player card revealed (flip sound)'
},
card_legendary_reveal: {
  variants: [{ id: 'clr', src: '/sounds/rewards/legendary_reveal.mp3' }],
  description: 'Special dramatic reveal for Legendary/Mythic cards — 2s buildup + reveal'
},
chest_unlock: {
```

```
    variants: [
      { id: 'cu1', src: '/sounds/rewards/chest_unlock_1.mp3' },
      { id: 'cu2', src: '/sounds/rewards/chest_unlock_2.mp3' },
      { id: 'cu3', src: '/sounds/rewards/chest_unlock_3.mp3' },
    ],
    description: 'Lock turning + lid opening + sparkle burst'
  },
  level_up: {
    variants: [{ id: 'lu', src: '/sounds/rewards/level_up.mp3' }],
    description: 'Triumphant fanfare for player level up'
  },

  // ── RUNNING BETWEEN WICKETS ──
  running_footsteps: {
    variants: [
      { id: 'rf1', src: '/sounds/field/running_1.mp3' },
      { id: 'rf2', src: '/sounds/field/running_2.mp3' },
    ],
    description: 'Batsmen running between wickets on grass'
  },

  // ── FIELDING ──
  fielder_dive: {
    variants: [
      { id: 'fd1', src: '/sounds/field/dive_1.mp3' },
    ],
    description: 'Fielder diving on grass (sliding sound)'
  },
  fielder_throw: {
    variants: [
      { id: 'ft1', src: '/sounds/field/throw_1.mp3' },
    ],
    description: 'Ball being thrown at stumps'
  },
};

// Total: 87 unique sound files across 38 categories
```

5.4 PERSPECTIVE AUDIO SYSTEM

The core innovation: the SAME game event produces DIFFERENT audio experiences for each player.

```
// In the EventEngine listener setup:

eventEngine.on('BOUNDARY_SIX', (payload) => {
  if (myRole === 'batting') {
    // I HIT the six — I feel euphoric
    soundEngine.playEffect('bat_hit_massive');
    setTimeout(() => soundEngine.playEffect('ball_flight_whoosh'), 200);
    setTimeout(() => soundEngine.playEffect('crowd_eruption'), 800);
    setTimeout(() => soundEngine.playEffect('firework_pop'), 1200);
    setTimeout(() => soundEngine.playEffect('firework_pop'), 1500);
    soundEngine.vibrate('heavy');
  } else {
    // OPPONENT hit a six against me — I feel deflated
    soundEngine.playEffect('bat_hit_massive'); // Same initial crack (you still hear the hit)
    setTimeout(() => soundEngine.playEffect('crowd_groan'), 800); // But crowd groans for ME
    // No fireworks for me
    // Commentary will be disappointed (handled by CommentaryEngine)
    soundEngine.vibrate('medium');
  }
});

eventEngine.on('WICKET_BOWLED', (payload) => {
```

```
  if (myRole === 'bowling') {
    // I TOOK the wicket — celebration!
    soundEngine.playEffect('stumps_hit');
    setTimeout(() => soundEngine.playEffect('crowd_appeal'), 300);
    setTimeout(() => soundEngine.playEffect('crowd_celebration_sustained'), 800);
    soundEngine.vibrate('heavy');
  } else {
    // MY batsman got out — devastating
    soundEngine.playEffect('stumps_hit'); // Same stump sound
    setTimeout(() => soundEngine.playEffect('crowd_gasp'), 300);
    // Quieter crowd, no celebration
    soundEngine.vibrate('error');
  }
});
```

PvP Implementation: In PvP mode, each player's SoundEngine runs independently on their own device. The EventEngine fires the same event type, but each player's sound listeners check `myRole` to determine which variant to play. This means Player A hears euphoria while Player B hears disappointment for the SAME ball — simultaneously.

CHAPTER 6: COMMENTARY ENGINE

6.1 PURPOSE

Dynamic, context-aware commentary that's never repetitive. Supports English, Hindi, and Hinglish. Has duo commentary (two voices). Adapts tone to match theme/situation.

6.2 ARCHITECTURE

```
// engines/CommentaryEngine.ts

export type CommentaryLanguage = 'english' | 'hindi' | 'hinglish';
export type CommentaryEmotion = 'neutral' | 'excited' | 'disappointed' | 'shocked' | 'dramatic' | 'calm';
export type CommentaryVoice = 'main' | 'color';

export interface CommentaryLine {
  id: string;
  text: string;
  voice: CommentaryVoice;
  emotion: CommentaryEmotion;
  rate: number;              // Speech rate (0.7=slow dramatic, 1.0=normal, 1.4=excited)
  pitch: number;             // 0.8-1.2
  delay: number;             // Ms delay before speaking
  followUp?: CommentaryLine; // Color commentator response
  minSituation?: string;     // Only play in certain match situations
  theme?: string;            // Only play in certain themes (gully, stadium, etc.)
}

export class CommentaryEngine {
  private language: CommentaryLanguage = 'english';
  private lastPlayedIds: string[] = [];           // Last 15 lines played (anti-repetition)
  private currentTheme: string = 'stadium';       // Affects tone/vocabulary
  private isMuted: boolean = false;
  private voiceMain: SpeechSynthesisVoice | null = null;
  private voiceColor: SpeechSynthesisVoice | null = null;
```

```
private isSpeaking: boolean = false;

constructor() {
  this.initVoices();
}

private async initVoices(): Promise<void> {
  // Wait for voices to load
  if (speechSynthesis.getVoices().length === 0) {
    await new Promise<void>(resolve => {
      speechSynthesis.onvoiceschanged = () => resolve();
    });
  }

  const voices = speechSynthesis.getVoices();

  // Prefer Indian English voice for main commentator
  this.voiceMain = voices.find(v => v.lang === 'en-IN')
    || voices.find(v => v.lang.startsWith('en'))
    || voices[0];

  // Color commentator: slightly different voice
  this.voiceColor = voices.find(v => v.lang === 'en-IN' && v !== this.voiceMain)
    || voices.find(v => v.lang === 'en-GB')
    || this.voiceMain;
}

/**
 * Get and speak a commentary line for an event.
 */
async speakForEvent(
  eventType: EventType,
  context: MatchContext,
  perspective: 'positive' | 'negative' | 'neutral'
): Promise<void> {
  if (this.isMuted || this.isSpeaking) return;

  const pool = this.getPool(eventType, perspective);
  if (!pool || pool.length === 0) return;

  // Filter: remove recently played lines
  const available = pool.filter(line => !this.lastPlayedIds.includes(line.id));
  if (available.length === 0) {
    // All lines exhausted — reset and use full pool
    this.lastPlayedIds = [];
    return this.speakForEvent(eventType, context, perspective);
  }

  // Filter by situation (some lines only work in tense/critical moments)
  const situationFiltered = available.filter(line => {
    if (!line.minSituation) return true;
    return line.minSituation === context.matchSituation;
  });

  const finalPool = situationFiltered.length > 0 ? situationFiltered : available;

  // Filter by theme
  const themeFiltered = finalPool.filter(line => {
    if (!line.theme) return true;
    return line.theme === this.currentTheme;
  });

  const pool2 = themeFiltered.length > 0 ? themeFiltered : finalPool;

  // Random selection
```

```
  const selected = pool2[Math.floor(Math.random() * pool2.length)];

  // Track for anti-repetition
  this.lastPlayedIds.push(selected.id);
  if (this.lastPlayedIds.length > 15) this.lastPlayedIds.shift();

  // Speak
  await this.speak(selected);

  // Speak follow-up (color commentator) if present
  if (selected.followUp) {
    await this.sleep(400); // Natural pause between commentators
    await this.speak(selected.followUp);
  }
}

private speak(line: CommentaryLine): Promise<void> {
  return new Promise((resolve) => {
    if (this.isMuted) { resolve(); return; }

    this.isSpeaking = true;

    const utterance = new SpeechSynthesisUtterance(line.text);
    utterance.voice = line.voice === 'main' ? this.voiceMain : this.voiceColor;
    utterance.rate = line.rate;
    utterance.pitch = line.pitch;
    utterance.volume = 1.0;

    if (this.language === 'hindi') {
      utterance.lang = 'hi-IN';
    } else {
      utterance.lang = 'en-IN';
    }

    utterance.onend = () => {
      this.isSpeaking = false;
      resolve();
    };

    utterance.onerror = () => {
      this.isSpeaking = false;
      resolve();
    };

    setTimeout(() => {
      speechSynthesis.speak(utterance);
    }, line.delay);
  });
}

/**
 * Also display the commentary text in the UI strip.
 */
getTextForUI(eventType: EventType, context: MatchContext, perspective: 'positive' | 'negative' | 'neutral'): string
  const pool = this.getPool(eventType, perspective);
  if (!pool || pool.length === 0) return '';
  const available = pool.filter(line => !this.lastPlayedIds.includes(line.id));
  const finalPool = available.length > 0 ? available : pool;
  return finalPool[Math.floor(Math.random() * finalPool.length)].text;
}

// ── CONFIGURATION ──

setLanguage(lang: CommentaryLanguage): void { this.language = lang; }
setTheme(theme: string): void { this.currentTheme = theme; }
```

```
  setMuted(muted: boolean): void { this.isMuted = muted; if (muted) speechSynthesis.cancel(); }

  private sleep(ms: number): Promise<void> {
    return new Promise(resolve => setTimeout(resolve, ms));
  }

  private getPool(eventType: EventType, perspective: string): CommentaryLine[] {
    // Import from language-specific data files
    // See Doc 3 for the complete 500+ line pools per language
    const data = this.language === 'english'
      ? englishCommentary
      : this.language === 'hindi'
        ? hindiCommentary
        : hinglishCommentary;

    const key = `${eventType}_${perspective}`;
    return data[key] || data[eventType] || [];
  }
}
```

6.3 COMMENTARY TRIGGERING RULES

Event	Speak?	Delay	Perspective for Batting Player	Perspective for Bowling Player
DOT_BALL	50% chance (don't speak every dot)	200ms	neutral	positive
RUNS_SCORED (1)	30% chance	100ms	neutral	neutral
RUNS_SCORED (2-3)	60% chance	100ms	positive	negative
BOUNDARY_FOUR	100% always	800ms (after bat crack sound)	excited	disappointed
BOUNDARY_SIX	100% always	1000ms (after initial sounds)	excited (rate 1.3)	disappointed
WICKET_*	100% always	500ms (dramatic pause first)	— (bowler's perspective)	— (batsman's perspective)
OVER_END	80% chance	300ms	neutral summary	neutral summary
INNINGS_END	100%	500ms	summary	summary
MILESTONE_50	100%	200ms	excited (for the batsman)	neutral
MILESTONE_100	100%	200ms	very excited	neutral
MATCH_END	100%	500ms	result-specific	result-specific
HATTRICK	100%	200ms	ecstatic	devastated

CRITICAL RULE: Commentary NEVER overlaps with itself. If a previous line is still being spoken, the new line waits or is skipped. The `isSpeaking` flag prevents any overlap.

CRITICAL RULE 2: Commentary NEVER plays during the suspense pause between picks and result. The 200-500ms pause after picks must be SILENT (just heartbeat/tension).

6.4 DUO COMMENTARY STRUCTURE

Every commentary line can optionally have a `followUp` from the color commentator:

```
Ball event happens →

Main commentator (0ms delay, rate 1.2):
  "FOUR! Driven beautifully through the covers!"

[400ms natural pause]

Color commentator (follows main, rate 1.0):
  "That's batting of the highest class. Middle of the bat."
```

Rules for duo:

- Main ALWAYS speaks first
- Color only speaks after main finishes + 400ms gap
- Not every line has a followUp — about 40% do
- Color commentator lines are shorter (< 8 words typically)
- Color adds analysis, emotion, or humor — never repeats what main said

6.5 THEME-ADAPTIVE COMMENTARY

The `currentTheme` modifies the commentary pool:

Theme	Tone Shift	Example Lines
`gully`	Street slang, trash talk, rageful, casual	"Arre bhai CHHAKKA! Gully ka raja!" / "Tera out ho gaya chal nikal!"
`classroom`	School humor, childish, principal references	"That six flew over the school wall — auto out-and-fetch!"
`school_ground`	Youthful, competitive, excitable	"BRO! That was INSANE! Best shot I've seen on this pitch!"
`beach`	Chill, relaxed, tropical	"Nice and easy… that's rolled all the way to the shoreline. Four."
`rooftop`	Urban, night vibes, intense	"Under the city lights, that six just woke up the whole neighborhood!"
`stadium`	Professional, broadcast quality	"Excellent use of the feet. Technically proficient stroke."
`street_night`	Underground, hype, intense	"OH! CHHAKKA! The streetlight is SHAKING from that hit!"
`village`	Rural, desi, earthy	"Waah ji waah! Pind da mundaa ne chakka maara!"
`ipl_arena`	Full broadcast, IPL commentary style	"That's gone into the stands! The IPL is all about ENTERTAINMENT!"
`world_cup`	High stakes, patriotic, historic	"And that's a SIX for India! The nation is on its feet!"

Theme-specific lines are ADDED to the pool, not replacing the generic ones. So in a gully match, you get 50 generic lines + 20 gully-specific lines = 70 total options for a SIX.

CHAPTER 7: LIGHTING ENGINE

7.1 PURPOSE

Controls all visual lighting effects: screen flashes, floodlight simulations, ambient mood, time-of-day, and event-specific effects. Implemented via CSS overlays and canvas layers.

7.2 IMPLEMENTATION

```
// engines/LightingEngine.ts

export class LightingEngine {
  private overlayElement: HTMLElement;    // Full-screen overlay div for flashes
  private vignetteElement: HTMLElement;   // Vignette overlay for tension
  private ambientBrightness: number = 1.0;

  constructor() {
    this.overlayElement = document.getElementById('lighting-overlay')!;
    this.vignetteElement = document.getElementById('vignette-overlay')!;
  }

  /**
   * Flash the entire screen a color for a brief moment.
   */
  flashScreen(color: string, durationMs: number, maxOpacity: number = 0.3): void {
    this.overlayElement.style.background = color;
    this.overlayElement.style.opacity = String(maxOpacity);
    this.overlayElement.style.transition = `opacity ${durationMs}ms ease-out`;

    requestAnimationFrame(() => {
      this.overlayElement.style.opacity = '0';
    });

    // Reset after transition
    setTimeout(() => {
      this.overlayElement.style.transition = '';
    }, durationMs + 50);
  }

  /**
   * Floodlight wave — simulates 4 floodlights flashing in sequence.
   * Used for SIX celebrations.
   */
  async floodlightWave(): Promise<void> {
    const positions = ['15%', '35%', '65%', '85%']; // Left positions of 4 virtual floodlights
    for (const pos of positions) {
      this.flashAt(pos, 'rgba(255,255,230,0.15)', 200);
      await this.sleep(120);
    }
  }

  private flashAt(xPos: string, color: string, durationMs: number): void {
    const flash = document.createElement('div');
    flash.style.cssText = `
      position: fixed; top: 0; left: ${xPos}; transform: translateX(-50%);
      width: 200px; height: 100vh;
      background: radial-gradient(ellipse at 50% 0%, ${color}, transparent 60%);
      pointer-events: none; z-index: 999; opacity: 1;
      transition: opacity ${durationMs}ms ease-out;
    `;
    document.body.appendChild(flash);
    requestAnimationFrame(() => { flash.style.opacity = '0'; });
```

```
    setTimeout(() => flash.remove(), durationMs + 50);
  }

  /**
   * Boundary rope glow — team-colored glow at the bottom of the pitch area.
   */
  glowBoundary(teamColor: string, durationMs: number = 1000): void {
    const glow = document.createElement('div');
    glow.style.cssText = `
      position: fixed; bottom: 35%; left: 0; right: 0; height: 4px;
      background: ${teamColor};
      box-shadow: 0 0 20px ${teamColor}, 0 0 40px ${teamColor};
      pointer-events: none; z-index: 998; opacity: 1;
      transition: opacity ${durationMs}ms ease-out;
    `;
    document.body.appendChild(glow);
    setTimeout(() => { glow.style.opacity = '0'; }, 100);
    setTimeout(() => glow.remove(), durationMs + 100);
  }

  /**
   * Set ambient brightness (for time of day, weather, match state).
   */
  setAmbientBrightness(brightness: number, fadeDuration: number = 500): void {
    this.ambientBrightness = brightness;
    document.documentElement.style.transition = `filter ${fadeDuration}ms ease`;
    document.documentElement.style.filter = `brightness(${brightness})`;
  }

  /**
   * Vignette effect — darkens screen edges for tense moments.
   */
  setVignette(intensity: number): void {
    // 0 = no vignette, 1 = maximum dark edges
    this.vignetteElement.style.background = `
      radial-gradient(ellipse at 50% 50%, transparent 50%, rgba(0,0,0,${intensity * 0.6}) 100%)
    `;
    this.vignetteElement.style.opacity = intensity > 0 ? '1' : '0';
  }

  private sleep(ms: number): Promise<void> {
    return new Promise(resolve => setTimeout(resolve, ms));
  }
}
```

7.3 LIGHTING EVENT MAP

Game Event	Lighting Effect	Details
DOT_BALL	None	—
RUNS_SCORED (1-3)	None	—
BOUNDARY_FOUR	`glowBoundary(teamColor, 1200)`	Team-colored rope glow for 1.2s
BOUNDARY_SIX	`flashScreen('#FFFFFF', 100, 0.25)` → `floodlightWave()` → firework light bursts (3-5)	Full sequence: white flash → wave → fireworks
WICKET_*	`flashScreen('#FF0000', 200, 0.2)` → `setAmbientBrightness(0.85, 500)` → restore after 1s	Red flash → slight dim → restore
POWERPLAY_START	`flashScreen(teamColor, 300, 0.15)`	Brief team-colored energy pulse

DEATH_OVERS_START	`setAmbientBrightness(1.05)`	Slightly brighter — intensity building
MATCH_END (win)	`setAmbientBrightness(1.1)` + continuous firework lights	Maximum brightness + celebration
MATCH_END (loss)	`setAmbientBrightness(0.6)` + blue-grey color grade	Dimmed + desaturated
Weather: rain	`setAmbientBrightness(0.8)`	Darker for rain
Weather: dust storm	Orange tint overlay (`rgba(210,180,140,0.15)`)	Warm haze
Match situation: tense	`setVignette(0.3)`	Subtle edge darkening
Match situation: critical	`setVignette(0.5)` + heartbeat sound	Stronger vignette
Last over of chase	`setVignette(0.4)` + floodlight intensity 1.1 + heartbeat	Maximum tension atmosphere

---

CHAPTER 8: WEATHER ENGINE

8.1 PURPOSE

Dynamic weather that affects BOTH visuals AND gameplay. Weather is assigned per match (random or theme-dependent).

8.2 IMPLEMENTATION

```
// engines/WeatherEngine.ts

import { tsParticles } from 'tsparticles';

export type WeatherState = 'clear' | 'overcast' | 'drizzle' | 'heavy_dew' | 'dust_storm' | 'night_lights' | 'golden_ho

export class WeatherEngine {
  private currentWeather: WeatherState = 'clear';
  private particleContainerId: string = 'weather-particles';

  async setWeather(weather: WeatherState): Promise<void> {
    // Clean up previous weather
    await this.clearParticles();

    this.currentWeather = weather;

    switch (weather) {
      case 'clear':
        lightingEngine.setAmbientBrightness(1.0);
        break;

      case 'overcast':
        lightingEngine.setAmbientBrightness(0.85);
        // Grey tint overlay
        break;

      case 'drizzle':
        lightingEngine.setAmbientBrightness(0.75);
```

```
      soundEngine.setAmbient('/sounds/ambient/rain_loop.mp3', 0.3);
      await this.startRainParticles(200);
      // Gameplay: boundary chance -10%
      gameplayEngine.applyModifier('boundary_chance', -0.10);
      break;

    case 'heavy_dew':
      // Visual: glistening grass texture (CSS filter: saturate + brightness on pitch)
      // Gameplay: 2nd innings bowling accuracy -15%
      if (matchStore.getState().innings === 2) {
        gameplayEngine.applyModifier('bowling_accuracy', -0.15);
      }
      break;

    case 'dust_storm':
      lightingEngine.setAmbientBrightness(0.8);
      soundEngine.setAmbient('/sounds/ambient/wind_loop.mp3', 0.25);
      await this.startDustParticles(150);
      // Gameplay: all stats -5%
      gameplayEngine.applyModifier('all_stats', -0.05);
      break;

    case 'night_lights':
      lightingEngine.setAmbientBrightness(0.7);
      // Floodlight glow cones visible
      // Gameplay: spin harder to read (+10% spin effectiveness)
      gameplayEngine.applyModifier('spin_accuracy', 0.10);
      break;

    case 'golden_hour':
      lightingEngine.setAmbientBrightness(1.05);
      // Warm orange tint overlay (rgba(255,165,0,0.05))
      break;
  }
}

private async startRainParticles(count: number): Promise<void> {
  await tsParticles.load({
    id: this.particleContainerId,
    options: {
      particles: {
        number: { value: count },
        color: { value: '#B0C4DE' },
        shape: { type: 'line' },
        size: { value: { min: 1, max: 3 } },
        move: {
          enable: true,
          direction: 'bottom',
          speed: { min: 15, max: 25 },
          angle: { value: 15, offset: 0 }, // Slightly angled rain
          straight: true,
        },
        opacity: { value: { min: 0.3, max: 0.6 } },
        life: { duration: { value: 0 }, count: 0 }, // Infinite
      },
      fullScreen: { enable: true, zIndex: 50 },
      detectRetina: true,
    }
  });
}

private async startDustParticles(count: number): Promise<void> {
  await tsParticles.load({
    id: this.particleContainerId,
    options: {
```

```
        particles: {
          number: { value: count },
          color: { value: '#D2B48C' },
          shape: { type: 'circle' },
          size: { value: { min: 1, max: 4 } },
          move: {
            enable: true,
            direction: 'right',
            speed: { min: 3, max: 8 },
            random: true,
            outModes: 'out',
          },
          opacity: { value: { min: 0.1, max: 0.4 } },
          wobble: { enable: true, speed: 2, distance: 10 },
        },
        fullScreen: { enable: true, zIndex: 50 },
      }
    });
  }

  private async clearParticles(): Promise<void> {
    const container = tsParticles.domItem(0);
    if (container) {
      container.destroy();
    }
  }

  /**
   * Randomly assign weather for a match based on theme.
   */
  static getRandomWeather(theme: string): WeatherState {
    const weights: Record<string, Record<WeatherState, number>> = {
      gully: { clear: 40, overcast: 20, dust_storm: 15, golden_hour: 20, drizzle: 5, heavy_dew: 0, night_lights: 0 },
      school_ground: { clear: 50, overcast: 20, golden_hour: 20, dust_storm: 5, drizzle: 5, heavy_dew: 0, night_lights
      stadium: { clear: 25, overcast: 15, night_lights: 30, golden_hour: 10, heavy_dew: 10, drizzle: 8, dust_storm: 2
      beach: { clear: 50, golden_hour: 30, overcast: 10, drizzle: 10, dust_storm: 0, heavy_dew: 0, night_lights: 0 },
      default: { clear: 35, overcast: 15, night_lights: 20, golden_hour: 15, drizzle: 8, heavy_dew: 5, dust_storm: 2 }
    };

    const themeWeights = weights[theme] || weights.default;
    const total = Object.values(themeWeights).reduce((a, b) => a + b, 0);
    let roll = Math.random() * total;

    for (const [weather, weight] of Object.entries(themeWeights)) {
      roll -= weight;
      if (roll <= 0) return weather as WeatherState;
    }

    return 'clear';
  }
}
```

8.3 WEATHER GAMEPLAY EFFECTS

Weather	Batting Modifier	Bowling Modifier	Special
Clear	None	None	—
Overcast	-5% power	+10% swing (pace)	Slightly darker
Drizzle	-10% boundary chance	-5% accuracy (wet ball)	Rain particles, sound
Heavy Dew	None (1st inn), +10% (2nd)	None (1st), -15% (2nd)	Glistening grass

Dust Storm	-5% all	-5% all	Dust particles, orange haze, wind sound
Night Lights	+5% power (see ball well)	+10% spin (harder to read)	Floodlight glow, insect sounds
Golden Hour	+5% power	-5% accuracy (sun in eyes)	Warm orange tint, long shadows

---

CHAPTER 9: CROWD ENGINE

9.1 PURPOSE

Controls the crowd's MOOD, which drives: ambient crowd volume, crowd visual animations (in 3D pitch view), and triggers crowd events (Mexican wave, chants).

9.2 IMPLEMENTATION

```
// engines/CrowdEngine.ts

export class CrowdEngine {
  private mood: number = 30;    // 0-100 (0 = silent, 100 = pandemonium)
  private moodDecayRate: number = 0.5; // Mood decays per second when nothing happens
  private decayInterval: NodeJS.Timeout | null = null;
  private mexicanWaveThreshold: number = 90;
  private lastMexicanWave: number = 0;

  start(): void {
    // Start natural mood decay (crowd gets bored if nothing exciting happens)
    this.decayInterval = setInterval(() => {
      this.mood = Math.max(0, this.mood - this.moodDecayRate);
      this.updateAudio();
    }, 1000);
  }

  stop(): void {
    if (this.decayInterval) clearInterval(this.decayInterval);
  }

  /**
   * Called by EventEngine after each game event.
   */
  reactToEvent(eventType: EventType, payload: Record<string, any>): void {
    const oldMood = this.mood;

    switch (eventType) {
      case 'DOT_BALL':
        this.mood = Math.max(0, this.mood - 3);
        break;
      case 'RUNS_SCORED':
        this.mood += (payload.runs || 1) * 4;
        break;
      case 'BOUNDARY_FOUR':
        this.mood = Math.min(100, this.mood + 25);
        break;
      case 'BOUNDARY_SIX':
        this.mood = 100; // ALWAYS max for a six
```

```
      break;
    case 'WICKET_BOWLED':
    case 'WICKET_CAUGHT':
    case 'WICKET_LBW':
    case 'WICKET_RUN_OUT':
    case 'WICKET_STUMPED':
    case 'WICKET_CAUGHT_BEHIND':
      this.mood = 85; // High but mixed excitement
      break;
    case 'MILESTONE_50':
      this.mood = Math.min(100, this.mood + 30);
      break;
    case 'MILESTONE_100':
      this.mood = 100;
      break;
    case 'HATTRICK':
      this.mood = 100;
      break;
    case 'OVER_END':
      this.mood = Math.max(20, this.mood - 5); // Slight dip between overs
      break;
    case 'DEATH_OVERS_START':
      this.mood = Math.max(60, this.mood); // Minimum 60 in death overs
      break;
  }

  this.mood = Math.max(0, Math.min(100, this.mood));

  // Check for Mexican Wave trigger
  if (this.mood >= this.mexicanWaveThreshold &&
      Date.now() - this.lastMexicanWave > 30000) { // Max once per 30 seconds
    this.triggerMexicanWave();
  }

  this.updateAudio();
  this.updateVisuals();
}

private updateAudio(): void {
  // Scale crowd ambient volume with mood
  const volume = this.mood / 100 * 0.5; // Max 0.5 (50% of ambient max)
  soundEngine.setAmbientVolume(volume, 300);

  // Switch ambient loop based on mood level
  if (this.mood < 20) {
    soundEngine.setAmbient('/sounds/ambient/crowd_quiet_loop.mp3');
  } else if (this.mood < 50) {
    soundEngine.setAmbient('/sounds/ambient/crowd_moderate_loop.mp3');
  } else if (this.mood < 80) {
    soundEngine.setAmbient('/sounds/ambient/crowd_loud_loop.mp3');
  }
  // Above 80: the one-shot crowd effects (eruption, celebration) handle it
}

private updateVisuals(): void {
  // Emit event for UI to update crowd visualization
  // (In 3D pitch view, crowd characters react based on mood)
  eventEngine.emit('CROWD_REACT', {
    mood: this.mood,
    intensity: this.mood > 80 ? 'pandemonium' :
               this.mood > 50 ? 'active' :
               this.mood > 20 ? 'moderate' : 'quiet',
  }, { blocking: false, duration: 0 });
}
```

```
  private triggerMexicanWave(): void {
    this.lastMexicanWave = Date.now();
    eventEngine.emit('CROWD_MEXICAN_WAVE', {}, { blocking: false, duration: 3000 });
    // UI: A wave animation sweeps around the stadium crowd
    // Sound: Rising cheer sound that travels left-to-right (stereo pan)
  }

  getMood(): number { return this.mood; }
  setMood(mood: number): void { this.mood = Math.max(0, Math.min(100, mood)); }
}
```

9.3 CROWD VISUAL STATES (for 3D Pitch View in Doc 3)

Mood Range	Crowd Visual State	Crowd Audio State
0-20	Quiet — seated, minimal movement, scattered arm movements	Near silence, whispers
20-40	Moderate — some clapping, occasional standing	Light applause, murmur
40-60	Active — many standing, flags starting to wave, some jumping	Audible cheering, clapping rhythm
60-80	Excited — most standing, flags waving, foam fingers up, some crowd surfing	Loud sustained cheering, chanting
80-100	Pandemonium — EVERYONE standing, arms up, flags/items thrown, Mexican waves, dancing	DEAFENING roar, horns, drums, sirens

---

CHAPTER 10: ENGINE MANAGER

10.1 PURPOSE

Initializes all engines and wires them together. This is the single entry point that the React app calls to start the engine system.

10.2 IMPLEMENTATION

```
// engines/EngineManager.ts

import { EventEngine } from './EventEngine';
import { SyncEngine } from './SyncEngine';
import { GameplayEngine } from './GameplayEngine';
import { SoundEngine } from './SoundEngine';
import { CommentaryEngine } from './CommentaryEngine';
import { LightingEngine } from './LightingEngine';
import { WeatherEngine } from './WeatherEngine';
import { CrowdEngine } from './CrowdEngine';

class EngineManager {
  event: EventEngine;
  sync: SyncEngine;
  gameplay: GameplayEngine;
  sound: SoundEngine;
  commentary: CommentaryEngine;
```

```
lighting: LightingEngine;
weather: WeatherEngine;
crowd: CrowdEngine;

constructor() {
  this.event = new EventEngine();
  this.sync = new SyncEngine();
  this.gameplay = new GameplayEngine();
  this.sound = new SoundEngine();
  this.commentary = new CommentaryEngine();
  this.lighting = new LightingEngine();
  this.weather = new WeatherEngine();
  this.crowd = new CrowdEngine();

  this.wireEngines();
}

private wireEngines(): void {
  const E = this.event;

  // ── Sound Engine listeners ──
  E.on('DOT_BALL', () => this.sound.playEffect('ball_into_gloves'));
  E.on('RUNS_SCORED', (p) => {
    this.sound.playEffect(p.runs <= 1 ? 'bat_hit_soft' : 'bat_hit_medium');
    if (p.runs >= 1) this.sound.playEffect('running_footsteps');
  });
  E.on('BOUNDARY_FOUR', () => {
    this.sound.playEffect('bat_hit_hard');
    setTimeout(() => this.sound.playEffect('crowd_cheer_excited'), 600);
  });
  E.on('BOUNDARY_SIX', () => {
    this.sound.playEffect('bat_hit_massive');
    setTimeout(() => this.sound.playEffect('ball_flight_whoosh'), 200);
    setTimeout(() => this.sound.playEffect('crowd_eruption'), 800);
    setTimeout(() => this.sound.playEffect('firework_pop'), 1200);
    setTimeout(() => this.sound.playEffect('firework_pop'), 1600);
    this.sound.vibrate('heavy');
  });
  E.on('WICKET_BOWLED', () => {
    this.sound.playEffect('stumps_hit');
    setTimeout(() => this.sound.playEffect('crowd_appeal'), 300);
    setTimeout(() => this.sound.playEffect('crowd_celebration_sustained'), 1000);
  });
  E.on('WICKET_CAUGHT', () => {
    this.sound.playEffect('ball_edge');
    setTimeout(() => this.sound.playEffect('crowd_gasp'), 200);
    setTimeout(() => this.sound.playEffect('crowd_celebration_sustained'), 1000);
  });
  E.on('WICKET_LBW', () => {
    this.sound.playEffect('ball_pad_hit');
    setTimeout(() => this.sound.playEffect('crowd_appeal'), 200);
    setTimeout(() => this.sound.playEffect('crowd_celebration_sustained'), 1500);
  });
  E.on('TOSS_COIN_FLIP', () => this.sound.playEffect('coin_flip'));
  E.on('TOSS_RESULT', () => this.sound.playEffect('coin_land'));
  E.on('MATCH_START', () => this.sound.playEffect('match_start_horn'));
  E.on('MATCH_END', (p) => {
    if (p.result === 'win') this.sound.playSpecific('/sounds/special/victory_fanfare.mp3');
    else this.sound.playSpecific('/sounds/special/defeat_sting.mp3');
  });
  E.on('UI_SHOW_CARD', () => this.sound.playEffect('ui_card_slide_in'));
  E.on('UI_HIDE_CARD', () => this.sound.playEffect('ui_card_slide_out'));
  E.on('UI_UPDATE_SCORE', () => this.sound.playEffect('ui_score_tick'));

  // ── Commentary Engine listeners ──
```

```
E.on('DOT_BALL', (p, e) => {
  if (Math.random() < 0.5) { // 50% chance to speak on dots
    this.commentary.speakForEvent('DOT_BALL', p.context, 'neutral');
  }
});
E.on('RUNS_SCORED', (p) => {
  if (p.runs >= 2 || Math.random() < 0.3) {
    this.commentary.speakForEvent('RUNS_SCORED', p.context, 'neutral');
  }
});
E.on('BOUNDARY_FOUR', (p) => {
  // Delay commentary to let the bat crack + crowd build first
  setTimeout(() => {
    this.commentary.speakForEvent('BOUNDARY_FOUR', p.context, 'excited');
  }, 800);
});
E.on('BOUNDARY_SIX', (p) => {
  setTimeout(() => {
    this.commentary.speakForEvent('BOUNDARY_SIX', p.context, 'excited');
  }, 1000);
});
E.on('WICKET_BOWLED', (p) => {
  setTimeout(() => {
    this.commentary.speakForEvent('WICKET_BOWLED', p.context, 'dramatic');
  }, 500); // Dramatic pause before commentary
});
E.on('WICKET_CAUGHT', (p) => {
  setTimeout(() => {
    this.commentary.speakForEvent('WICKET_CAUGHT', p.context, 'dramatic');
  }, 800);
});
E.on('WICKET_LBW', (p) => {
  setTimeout(() => {
    this.commentary.speakForEvent('WICKET_LBW', p.context, 'dramatic');
  }, 1200); // Extra delay for umpire finger raise
});
E.on('MILESTONE_50', (p) => {
  this.commentary.speakForEvent('MILESTONE_50', p.context, 'excited');
});
E.on('MILESTONE_100', (p) => {
  this.commentary.speakForEvent('MILESTONE_100', p.context, 'excited');
});
E.on('OVER_END', (p) => {
  if (Math.random() < 0.8) {
    this.commentary.speakForEvent('OVER_END', p.context, 'neutral');
  }
});

// ── Lighting Engine listeners ──
E.on('BOUNDARY_FOUR', () => {
  this.lighting.glowBoundary('var(--team-accent)', 1200);
});
E.on('BOUNDARY_SIX', () => {
  this.lighting.flashScreen('#FFFFFF', 100, 0.25);
  setTimeout(() => this.lighting.floodlightWave(), 200);
});
E.on('WICKET_BOWLED', () => this.lighting.flashScreen('#FF0000', 200, 0.2));
E.on('WICKET_CAUGHT', () => this.lighting.flashScreen('#FF0000', 200, 0.2));
E.on('WICKET_LBW', () => this.lighting.flashScreen('#FF0000', 200, 0.2));
E.on('WICKET_RUN_OUT', () => this.lighting.flashScreen('#FF0000', 200, 0.2));
E.on('DEATH_OVERS_START', () => this.lighting.setAmbientBrightness(1.05));

// ── Crowd Engine listeners ──
const crowdEvents: EventType[] = [
  'DOT_BALL', 'RUNS_SCORED', 'BOUNDARY_FOUR', 'BOUNDARY_SIX',
```

```
      'WICKET_BOWLED', 'WICKET_CAUGHT', 'WICKET_LBW', 'WICKET_RUN_OUT',
      'WICKET_STUMPED', 'WICKET_CAUGHT_BEHIND',
      'MILESTONE_50', 'MILESTONE_100', 'HATTRICK',
      'OVER_END', 'DEATH_OVERS_START'
    ];
    crowdEvents.forEach(type => {
      E.on(type, (payload) => this.crowd.reactToEvent(type, payload));
    });

    // ── Event logging (for replay system) ──
    E.onAny((payload, event) => {
      // Store in match store for replay
      // matchStore.getState().addToEventLog(event);
    });
  }

  /**
   * Initialize engines for a new match.
   */
  initMatch(settings: { weather: WeatherState; theme: string; language: CommentaryLanguage }): void {
    this.weather.setWeather(settings.weather);
    this.commentary.setLanguage(settings.language);
    this.commentary.setTheme(settings.theme);
    this.crowd.start();
    this.crowd.setMood(30); // Start with moderate crowd energy
    this.sound.setAmbient('/sounds/ambient/crowd_quiet_loop.mp3');
  }

  /**
   * Cleanup after match ends.
   */
  cleanup(): void {
    this.crowd.stop();
    this.sound.stopAllEffects();
    this.event.clearQueue();
  }
}

// Singleton
export const engines = new EngineManager();
```

10.3 USAGE IN REACT COMPONENTS

```
// In a React component:
import { engines } from '@/engines/EngineManager';

// When player taps a hand number during match:
function onHandSelected(pick: number) {
  // 1. Disable buttons
  setButtonsDisabled(true);

  // 2. Send pick to sync engine (PvP) or resolve locally (vs AI)
  if (isPvP) {
    engines.sync.sendBallPick(pick, 'batting', currentBallId);
    // Wait for SyncEngine to broadcast result
  } else {
    // AI mode: resolve immediately
    const aiPick = Math.floor(Math.random() * 6) + 1;
    const result = engines.gameplay.resolveBall(pick, aiPick, batsman, bowler, context);

    // Fire the event sequence
    engines.event.emit('BALL_BOWLER_RUNUP', {});
    // After runup duration, fire result
```

```
    setTimeout(() => {
      if (result.isBoundarySix) {
        engines.event.emit('BOUNDARY_SIX', { ...result, context });
      } else if (result.isBoundaryFour) {
        engines.event.emit('BOUNDARY_FOUR', { ...result, context });
      } else if (result.isWicket) {
        engines.event.emit(`WICKET_${result.dismissalType!.toUpperCase()}`, { ...result, context });
      } else if (result.isDot) {
        engines.event.emit('DOT_BALL', { ...result, context });
      } else {
        engines.event.emit('RUNS_SCORED', { ...result, context });
      }
    }, 600); // After bowler runup duration
  }
}
```

CHAPTER 11: HTML OVERLAY ELEMENTS

The Lighting Engine and Weather Engine need DOM elements to function. Add these to your root layout:

```
<!-- In index.html or root layout component -->
<div id="app-root">
  <!-- Your React app renders here -->
</div>

<!-- Engine overlay layers (above the app, below modals) -->
<div id="lighting-overlay" style="
  position: fixed; top: 0; left: 0; right: 0; bottom: 0;
  pointer-events: none; z-index: 900; opacity: 0;
  transition: opacity 0.1s ease;
"></div>

<div id="vignette-overlay" style="
  position: fixed; top: 0; left: 0; right: 0; bottom: 0;
  pointer-events: none; z-index: 899; opacity: 0;
  transition: opacity 0.5s ease;
"></div>

<div id="weather-particles" style="
  position: fixed; top: 0; left: 0; right: 0; bottom: 0;
  pointer-events: none; z-index: 898;
"></div>
```

This Doc 2 covers all 8 game engines with full TypeScript implementations, complete sound library (87 files across 38 categories), ball resolution algorithm with special abilities and modifiers, PvP sync protocol with sync points and timeout handling, commentary triggering rules, lighting effect map, weather particle systems with gameplay modifiers, crowd mood system with Mexican wave triggers, and the EngineManager that wires everything together. The complete ball sequence is documented millisecond-by-millisecond. Commentary content (the actual 500+ lines per language) is in Doc 3. Feed this alongside Doc 1 to the builder.